



Redes Neurais

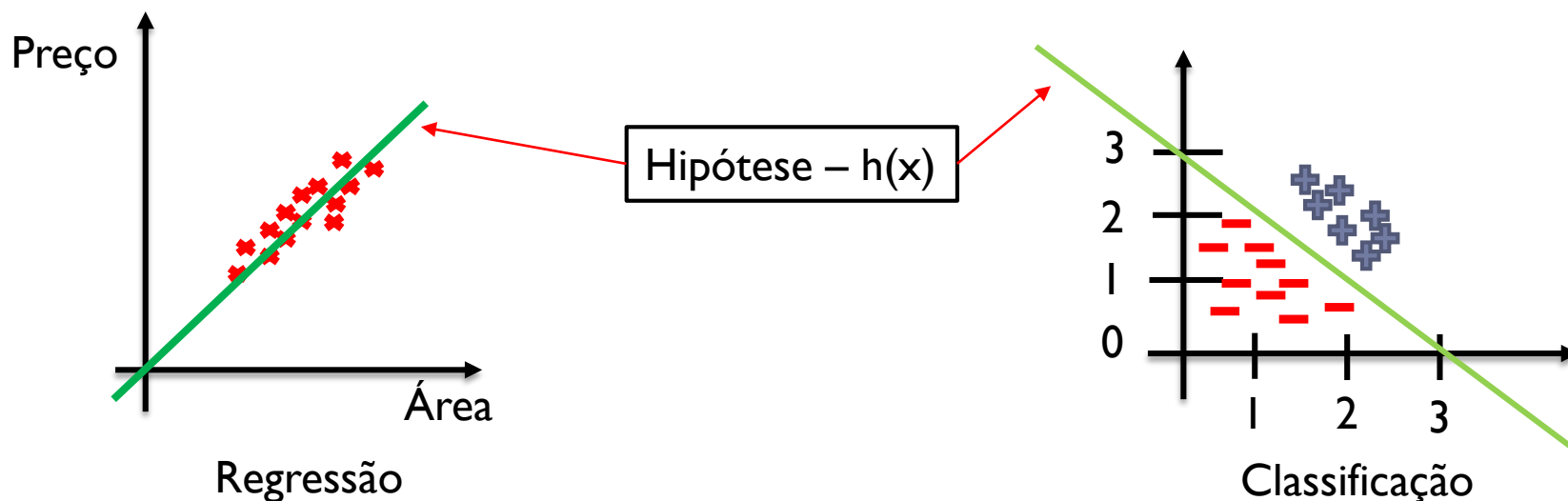
Profs. Bruno Nogueira e Edson Takashi

Agradecimentos

- ▶ Parte dos slides desta aula são baseados no material do prof. Andrew Ng

Aprendizado de Máquina Supervisionado

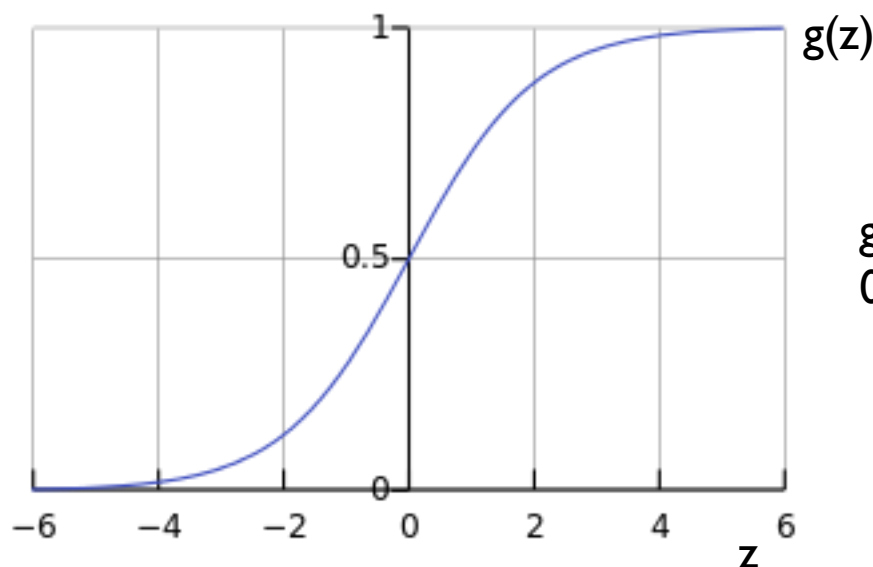
- ▶ Aprender uma função $h(x)$ que se aproxime de uma função $f(x)$ que descreve o comportamento dos dados
 - ▶ Aprendizado a partir de um conjunto de dados (dados de treinamento)
 - ▶ Hipótese aprendida deve ser generalizável (aplicável a dados não vistos)



Regressão Logística

- ▶ O objetivo da regressão logística é fornecer classes discretas: $0 \leq h(x) \leq 1$
- ▶ Para isso, utiliza da função logística ou sigmoide

$$g(z) = \frac{1}{1 + e^{-z}} \quad z \text{ é um número real}$$

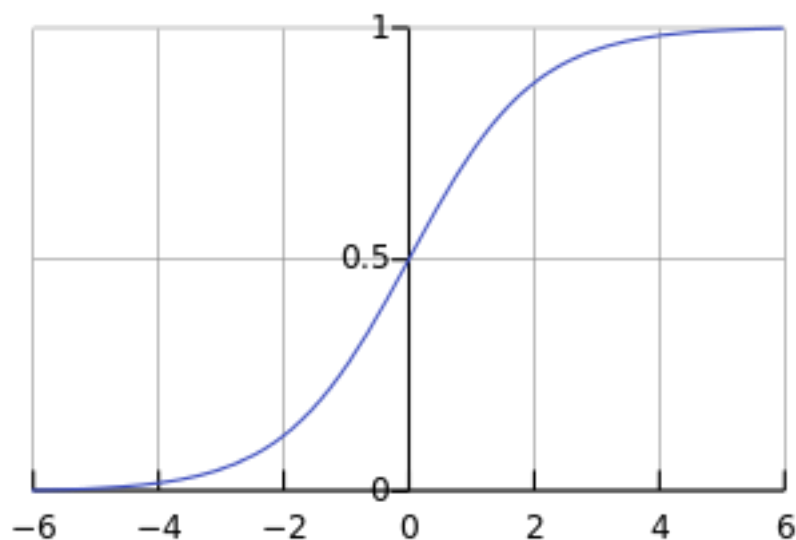


$g(z)$ nunca toca em 1 ou 0 (tende ao infinito)

Regressão Logística

- Trazendo a função logística para a saída do modelo, temos:

$$h(x) = g(\theta^T x) \quad \longrightarrow \quad h(x) = \frac{1}{1 + e^{-\theta^T x}}$$



Regressão logística como probabilidade

- ▶ Podemos interpretar $h(x)$ como sendo a probabilidade estimada de $y = 1$ para a entrada x
 - ▶ Por exemplo, no problema dos cogumelos, se $h(x) = 0,7$, interpretamos que há 70% de chances de ser comestível
- ▶ Formalmente:

$$h(x) = P(y = 1|x; \theta)$$

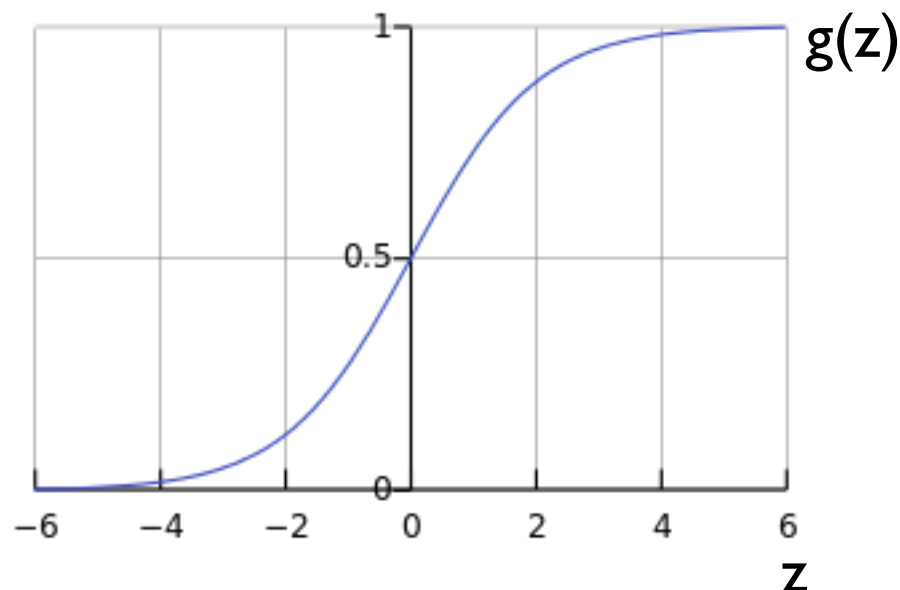
$$P(y = 0|x; \theta) + P(y = 1|x; \theta) = 1$$

$$P(y = 0|x; \theta) = 1 - P(y = 1|x; \theta)$$

Borda de decisão

$$h(x) = g(\theta^T x)$$

$$g(z) = \frac{1}{1 + e^{-z}}$$



Predizer $y = 1$ se $h(x) \geq 0,5$

$$g(z) \geq 0,5$$

$$z \geq 0$$

$$\therefore \theta^T x \geq 0$$

Predizer $y = 0$ se $h(x) < 0,5$

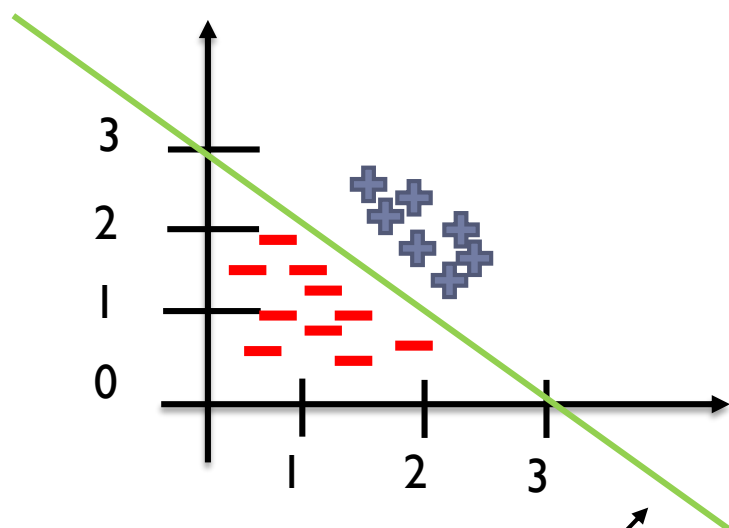
$$g(z) < 0,5$$

$$z < 0$$

$$\therefore \theta^T x < 0$$

Borda de decisão

- ▶ Os limiares impostos delimitam a separação entre as classes



Borda de Decisão

$$h(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

$$\theta = \begin{bmatrix} -3 \\ 1 \\ 1 \end{bmatrix}$$

Predizer $y = 1$ se $-3 + x_1 + x_2 \geq 0$

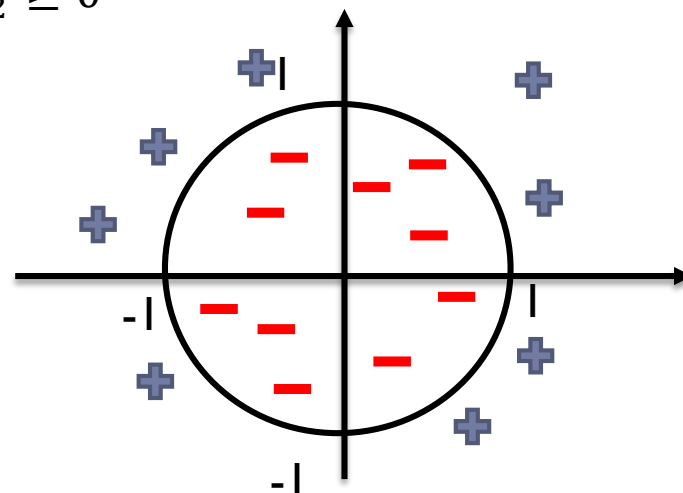
Predizer $y = 1$ se $x_1 + x_2 \geq 3$

Borda de decisão

- ▶ Desafio é achar os parâmetros que melhor se ajustem aos dados
- ▶ Termos polinomiais tornam a borda de decisão mais complexa

$$h(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2)$$

$$\theta = \begin{bmatrix} -1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix} \quad \begin{array}{l} \text{Predizer } y = 1 \text{ se } -1 + x_1^2 + x_2^2 \geq 0 \\ \text{Predizer } y = 1 \text{ se } x_1^2 + x_2^2 \geq 1 \end{array}$$



Borda de decisão

- ▶ Modelos mais complexos tendem a sofrer problemas de *overfitting*
 - ▶ Alta variância
- ▶ Modelos muito simples tendem a não se adequarem aos dados e sofrer *underfitting*
 - ▶ Alto bias
- ▶ Tem-se que levar em consideração o balando bias-variância
- ▶ **Navalha de Ockham**
 - ▶ Optar, dentre os modelos que mais se adequam aos dados, por aquele que tem a menor complexidade

Regressão Logística

Função de custo

- Vamos assumir um conjunto de treinamento com m exemplos

$$\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots (x^{(m)}, y^{(m)}), \}$$

$$x \in \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix}, x_0 = 1, y \in \{0, 1\}$$

$$h(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Regressão Logística

Função de custo

- Para regressão logística, definimos a função de custo como:

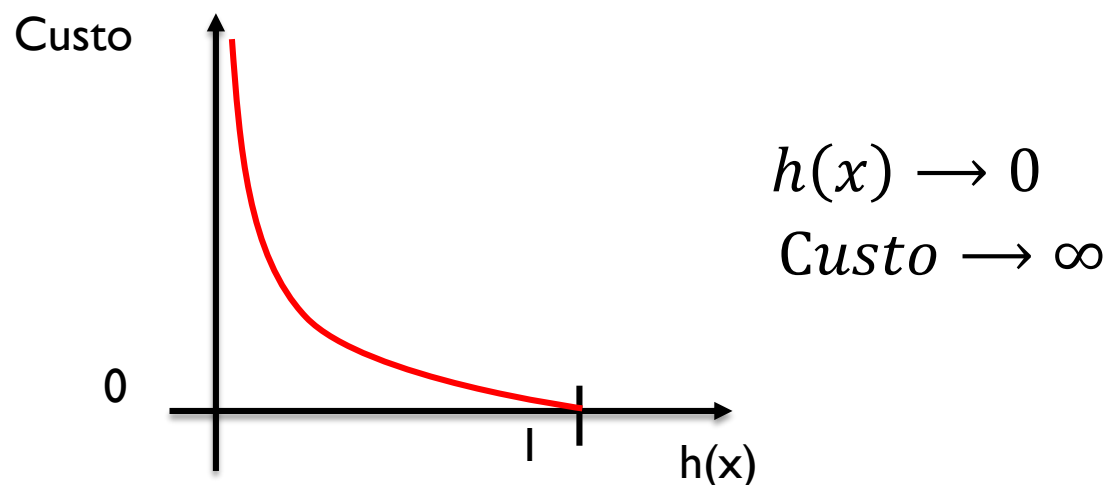
$$\text{Custo}(h(x), y) = \begin{cases} -\log(h(x)), & y = 1 \\ -\log(1 - h(x)), & y = 0 \end{cases}$$

Regressão Logística

Função de custo

- Para regressão logística, definimos a função de custo como:

$$Custo(h(x), y) = \begin{cases} -\log(h(x)), & y = 1 \\ -\log(1 - h(x)), & y = 0 \end{cases}$$



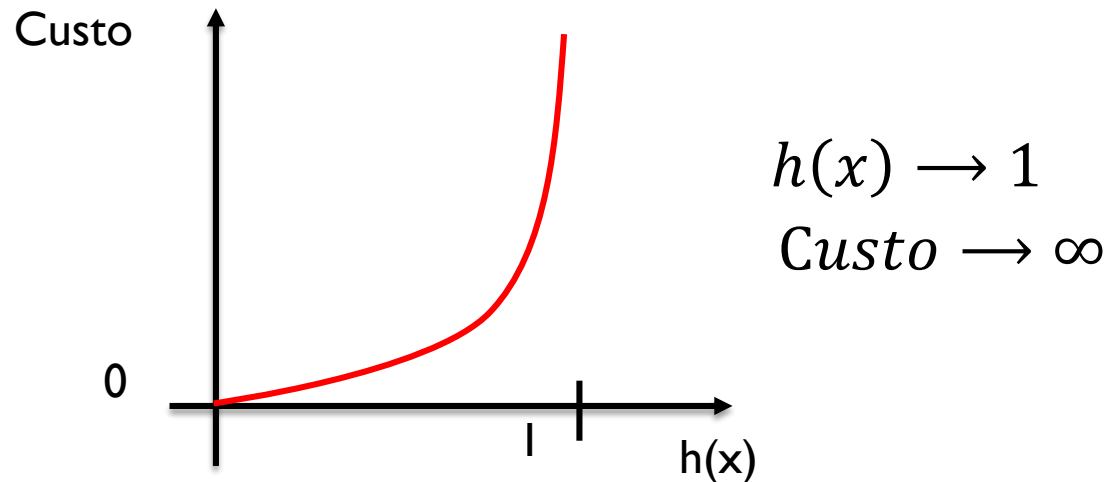
Regressão Logística

Função de custo



- Para regressão logística, definimos a função de custo como:

$$Custo(h(x), y) = \begin{cases} -\log(h(x)), & y = 1 \\ -\log(1 - h(x)), & y = 0 \end{cases}$$



Regressão Logística

Função de custo

- ▶ Podemos reescrever a função de custo para regressão logística de maneira unida:

$$Custo(h(x), y) = -y * \log(h(x)) - (1 - y) * \log(1 - h(x))$$

- ▶ Dessa forma, podemos reescrever a função objetivo:

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m Custo(h(x^{(i)}) - y^{(i)})$$

$$J(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} * \log(h(x^{(i)})) + (1 - y^{(i)}) * \log(1 - h(x^{(i)})) \right]$$

Função logística

$$J(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} * \log(h(x^{(i)})) + (1 - y^{(i)}) * \log(1 - h(x^{(i)})) \right]$$

- ▶ Objetivo: $\min_{\theta} J(\theta)$
- ▶ Para predição: $h(x) = \frac{1}{1 + e^{-\theta^T x}}$

Como encontrar θ

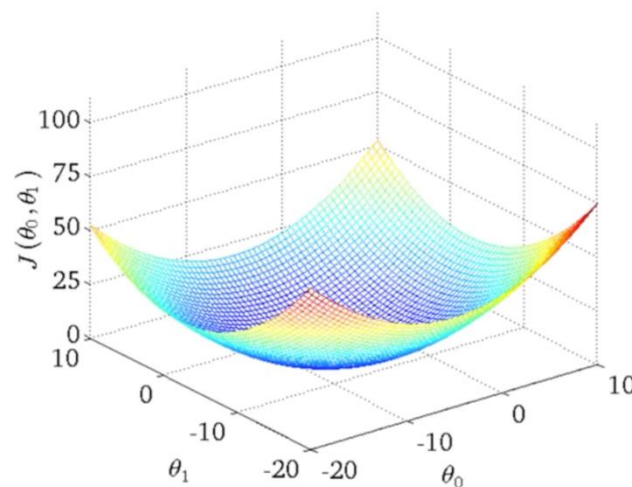
► Gradiente descendente!

ALGORITMO DO GRADIENTE DESCENDENTE

Repetir até convergência:

$$\theta_j := \theta_j - \alpha * \frac{1}{m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)}) * x_j^{(i)}$$

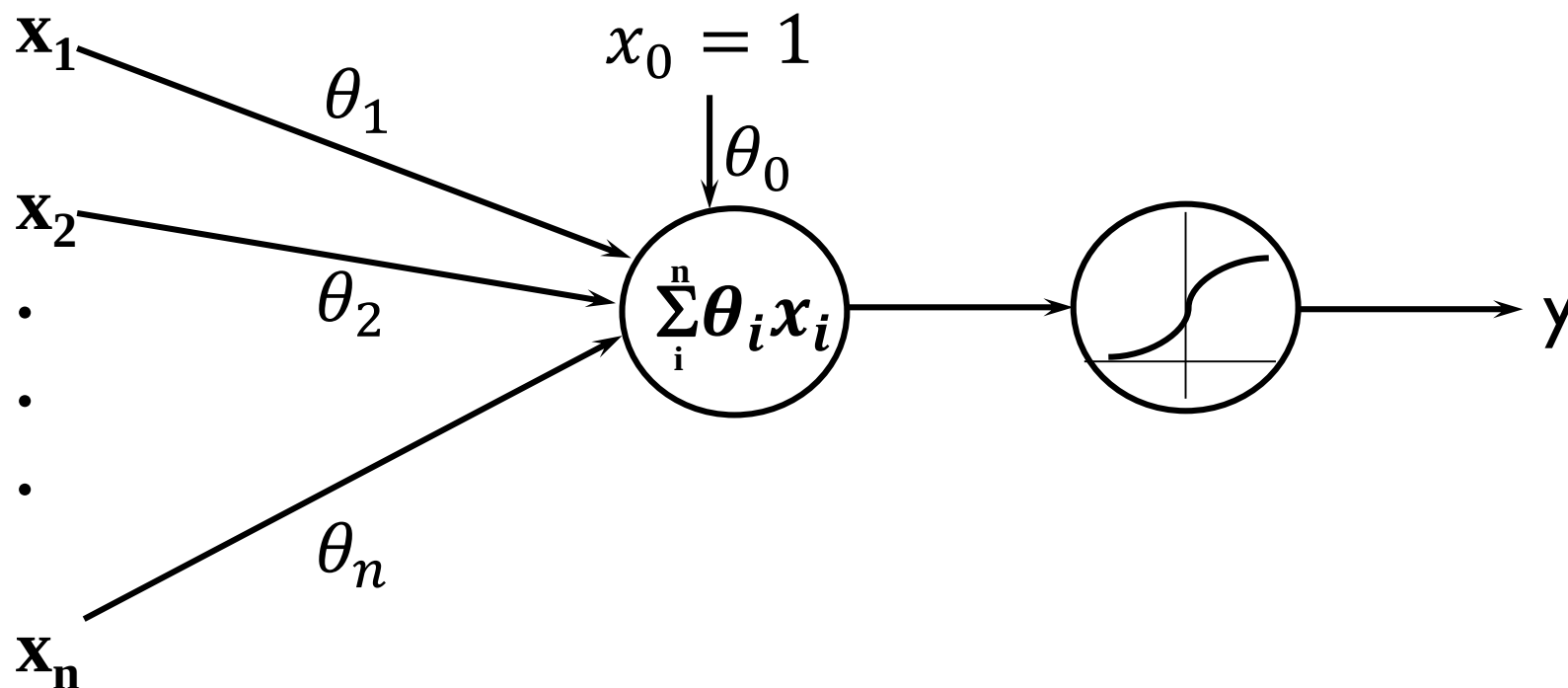
► $h(x) = \frac{1}{1 + e^{-\theta^T x}}$



Fonte: notas de aula do Prof. Andrew Ng

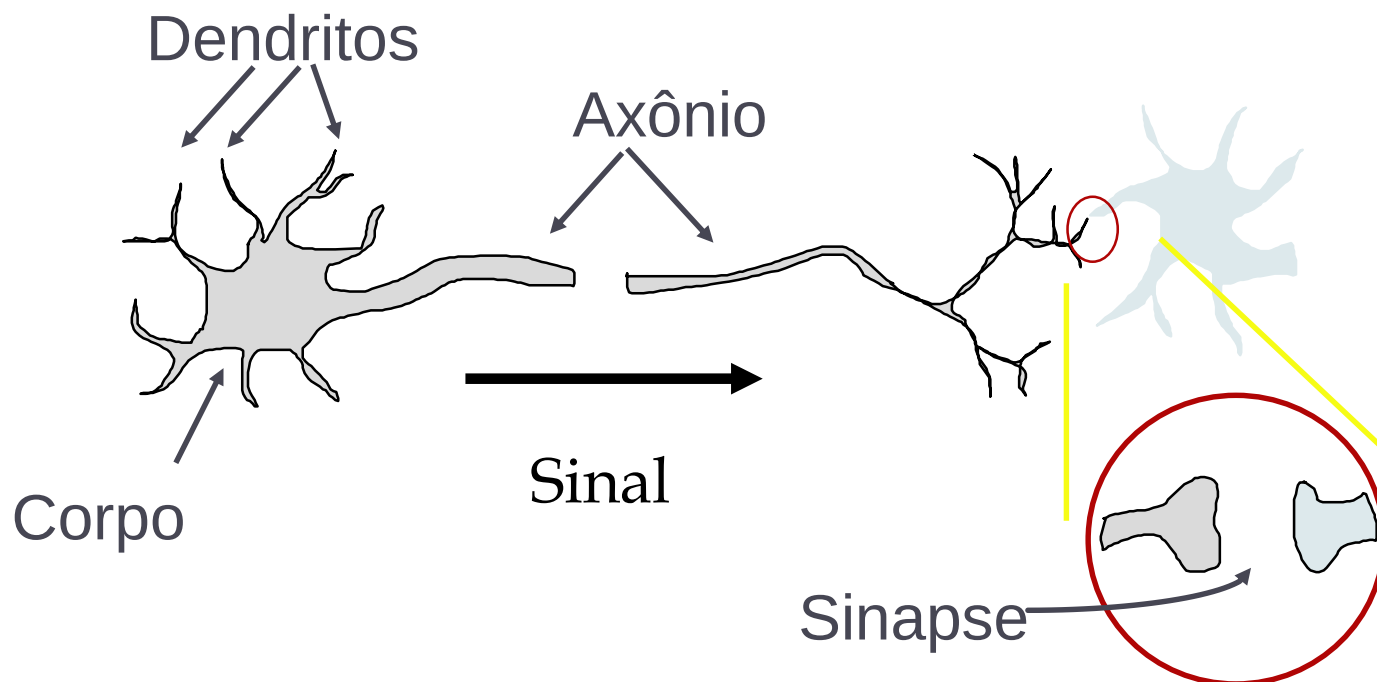
Unidade sigmoide

- ▶ Em redes neurais, um regressor logístico é conhecido como unidade sigmoide



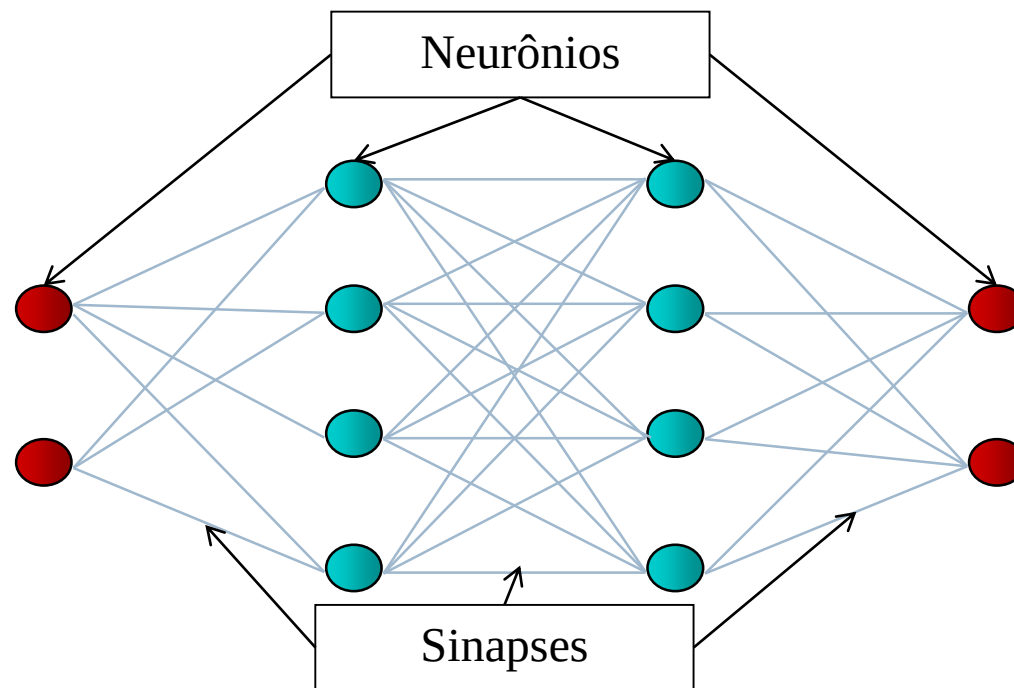
Mas afinal... O que são redes neurais?

- ▶ Modelos inspirados no cérebro humano
 - ▶ Compostos por unidades de processamento (**neurônios**)
 - ▶ Interligadas por um grande número de conexões (**sinapses**)



Redes Neurais Artificiais

- ▶ Modelos inspirados no cérebro humano
 - ▶ Compostos por unidades de processamento (**neurônios**)
 - ▶ Interligadas por um grande número de conexões (**sinapses**)

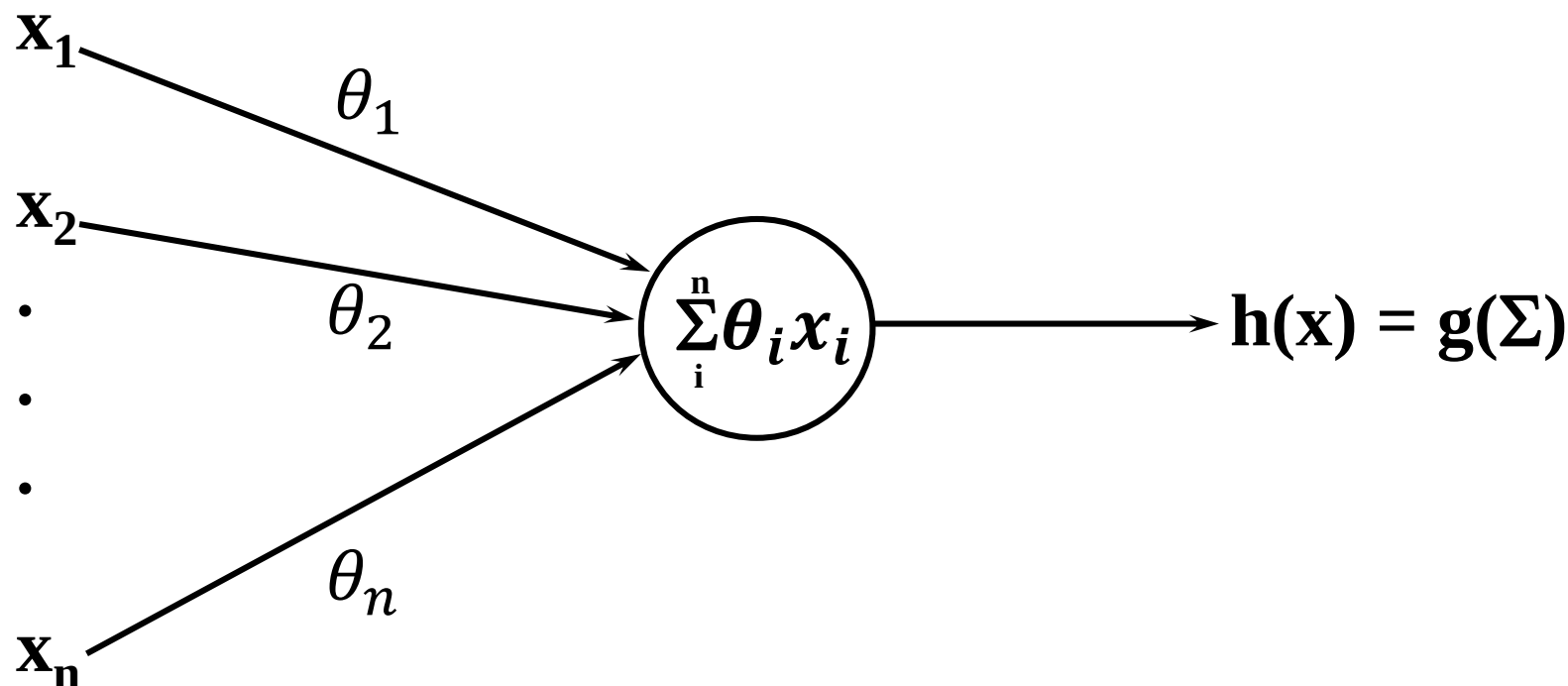


Redes Neurais Artificiais

- ▶ Tentativas de sistemas de aprendizado biologicamente realistas, baseadas em modelos abstratos sobre o cérebro
- ▶ Na prática, funcionam de forma inteiramente diferente
 - ▶ Neurônios são $\approx$ 1 milhão de vezes mais lentos que as portas lógicas de silício
 - ▶ Lentidão compensada por grande número de neurônios massivamente conectados
 - ▶ Para certas operações, o cérebro humano é muito mais rápido que computadores convencionais
 - ▶ Visão, audição, controle, previsão

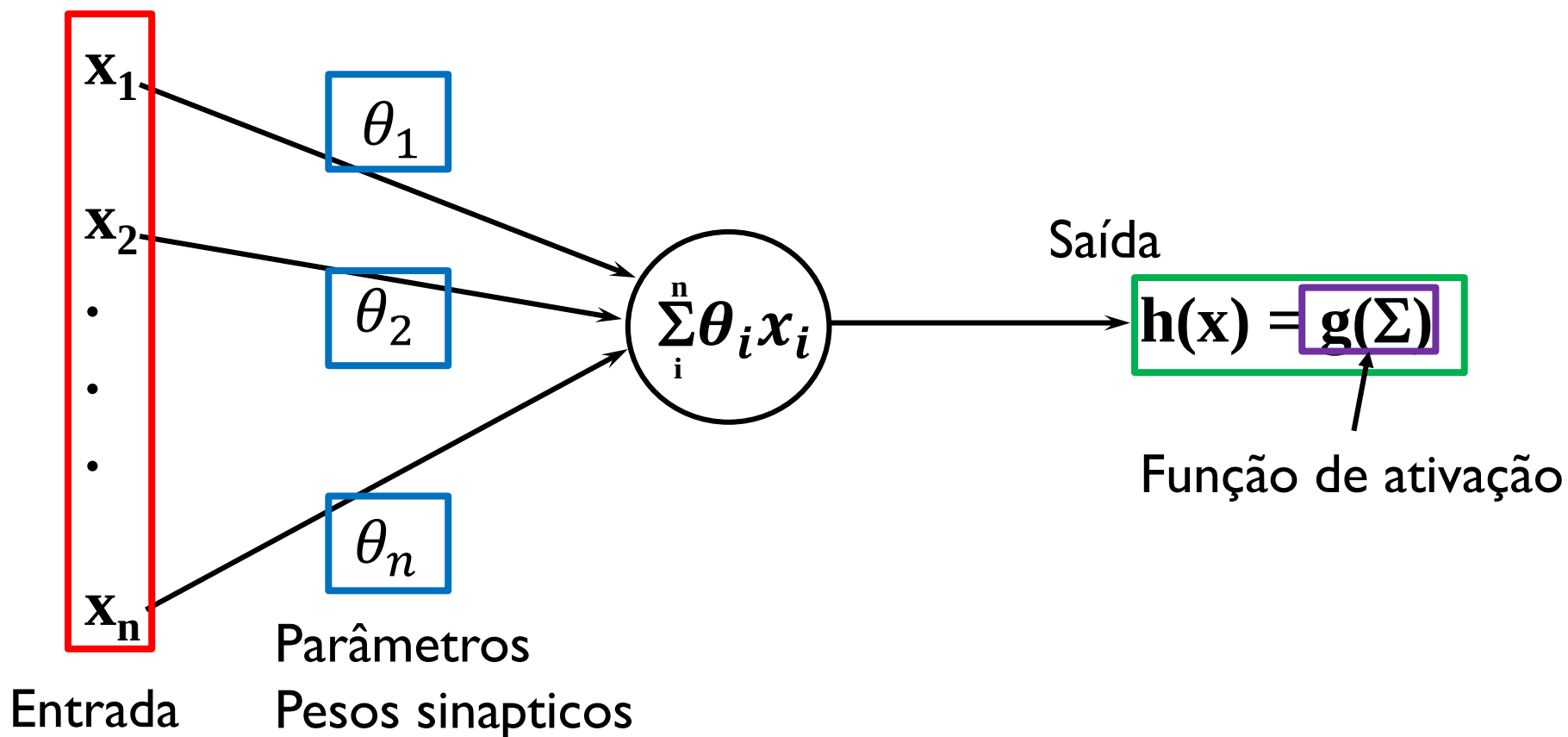
Neurônio artificial

- ▶ Também conhecido como unidade de processamento

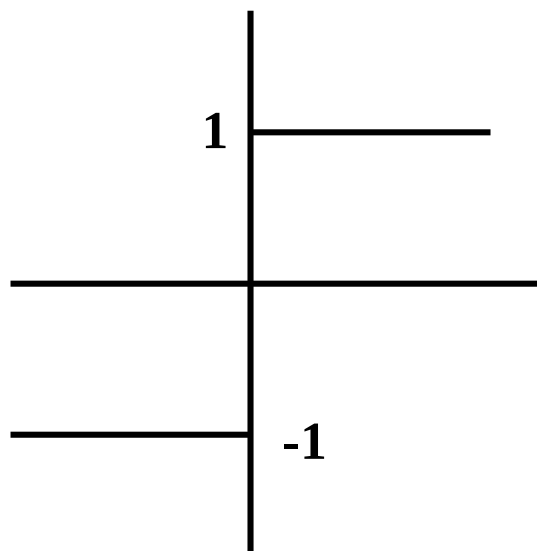


Neurônio artificial

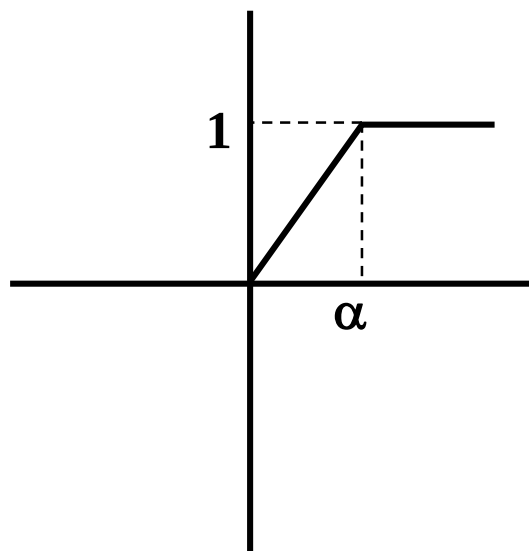
- ▶ Também conhecido como unidade de processamento



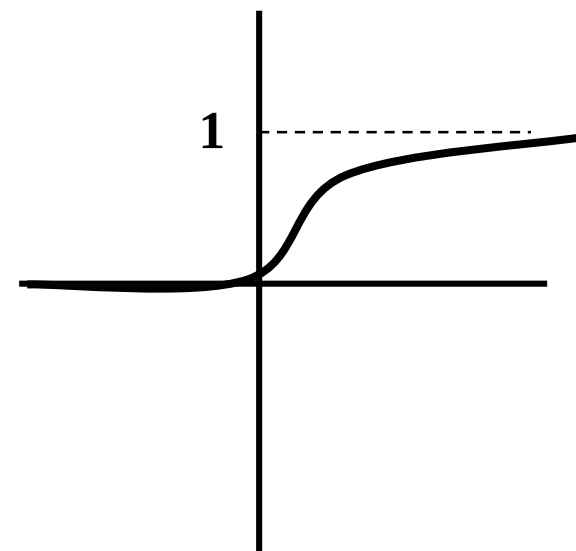
Funções de ativação



Hard Limiter
Degrau



Threshold
Logic



Sigmoid

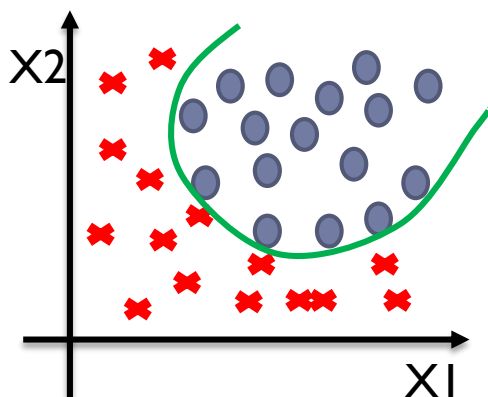
Voltando à unidade sigmoide

- ▶ Um regressor logístico é uma unidade sigmoide
- ▶ Geralmente, é o mais utilizado
- ▶ Tem a limitação de conseguir resolver problemas apenas linearmente separáveis. Como resolver?
 1. **Polinômio de maior grau**
 2. **Combinação de regressores logísticos**

Classificação não-linear

Polinômio de maior grau

- ▶ Funciona bem para poucos atributos
- ▶ Exemplo: termos quadráticos (segunda ordem)



$$\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2$$

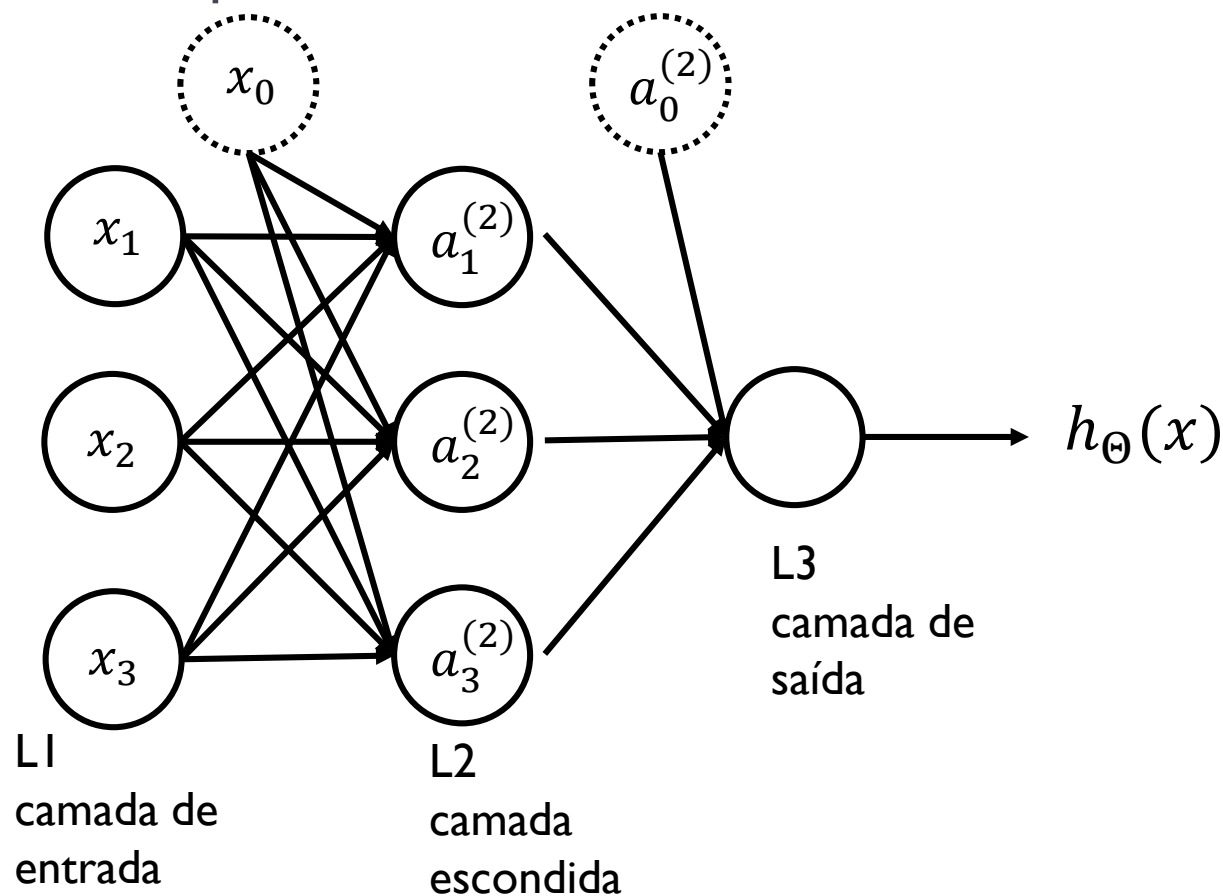
- ▶ Com $n = 100$, tem-se um polinômio de ≈ 5000 termos - $O(n^2)$
- ▶ Com um polinômio de grau 3, tem-se ≈ 170000 termos - $O(n^3)$
- ▶ **Computacionalmente muito custoso para muitos atributos**

Classificação não-linear

Combinação de unidades logísticas

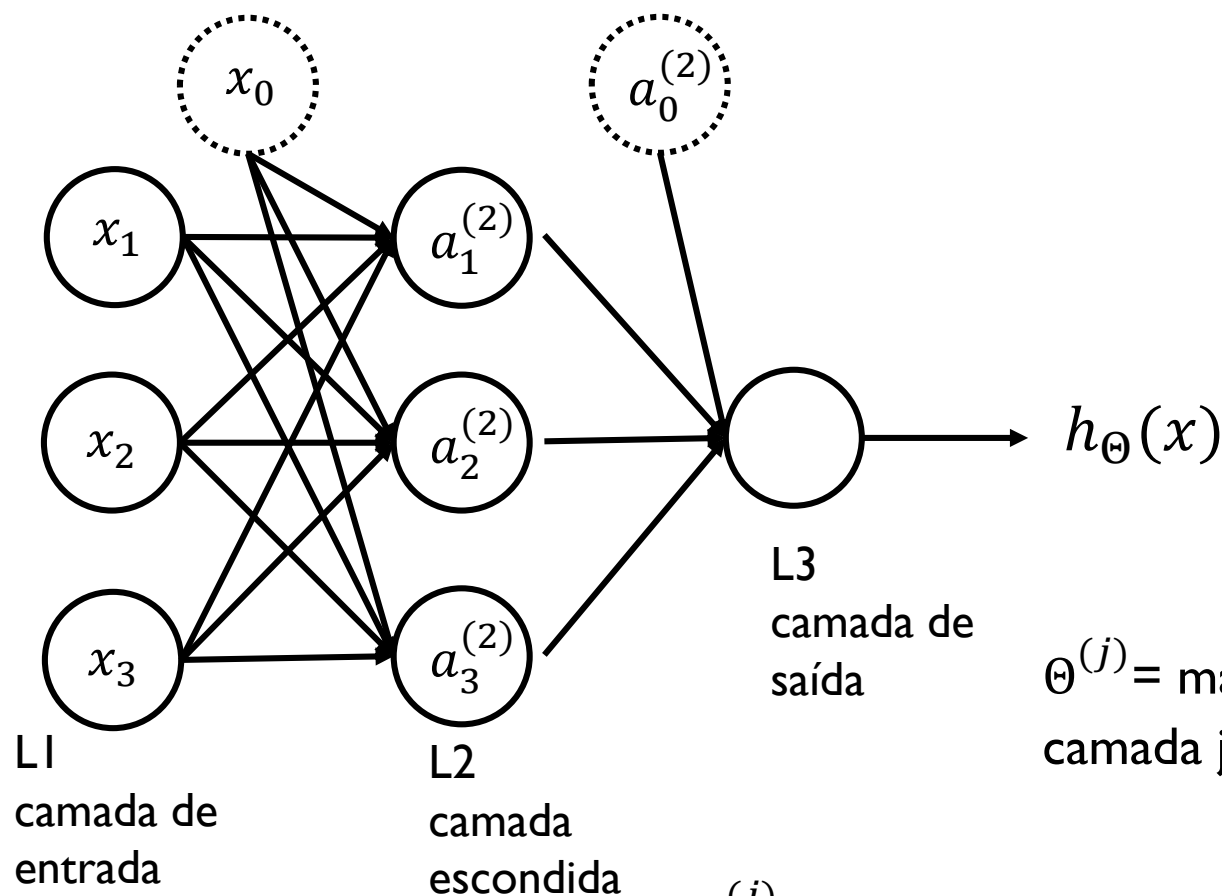
► Redes neurais

- Neurônios dispostos em diversas “camadas”



Classificação não-linear

Combinação de unidades logísticas



$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

$$h(x) = \frac{1}{1 + e^{-\theta^T x}}$$

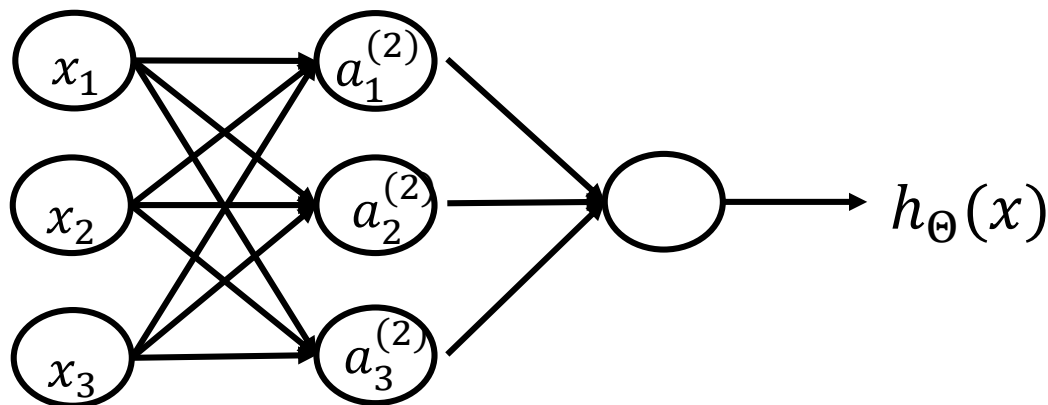
$\Theta^{(j)}$ = matriz de parâmetros ligando a camada j à camada $j+1$

$a_i^{(j)}$ = ativação do nó i na camada j
(saída da função de ativação daquele nó)

Matriz de parâmetros

- ▶ Cada matriz $\Theta^{(j)}$
 - ▶ Número de colunas = número de unidades da próxima camada (s_{j+1})
 - ▶ Número de linhas = número de unidades da camada atual + 1 (devido à unidade de bias) ($s_j + 1$)
 - ▶ $\Theta^{(j)} = [s_j + 1 \times s_{j+1}]$
 - ▶ Se tivermos $s_j = 4$ e $s_{j+1} = 3$, teremos
 - ▶ $\Theta^{(j)} = [5 \times 3]$

Valores de ativação



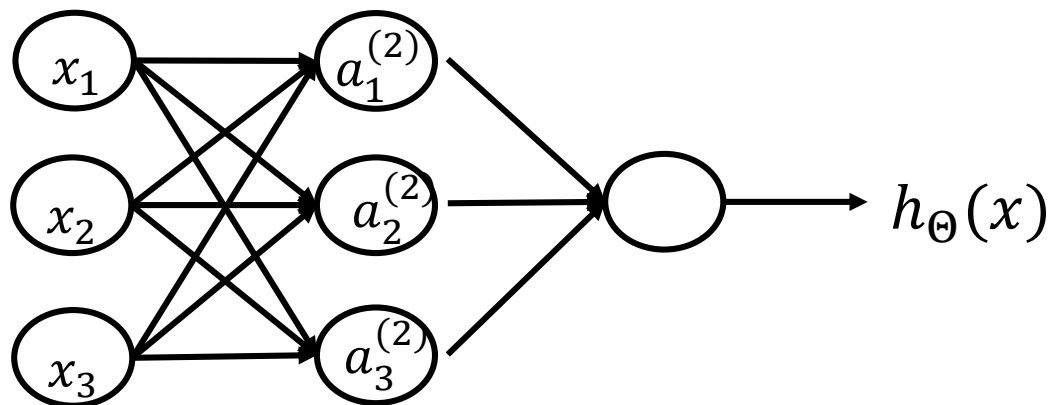
$$a_1^{(2)} = g(\Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3)$$

$$a_2^{(2)} = g(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3)$$

$$a_3^{(2)} = g(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3)$$

$$h_{\Theta}(x) = a_1^{(3)} = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)})$$

Valores de ativação

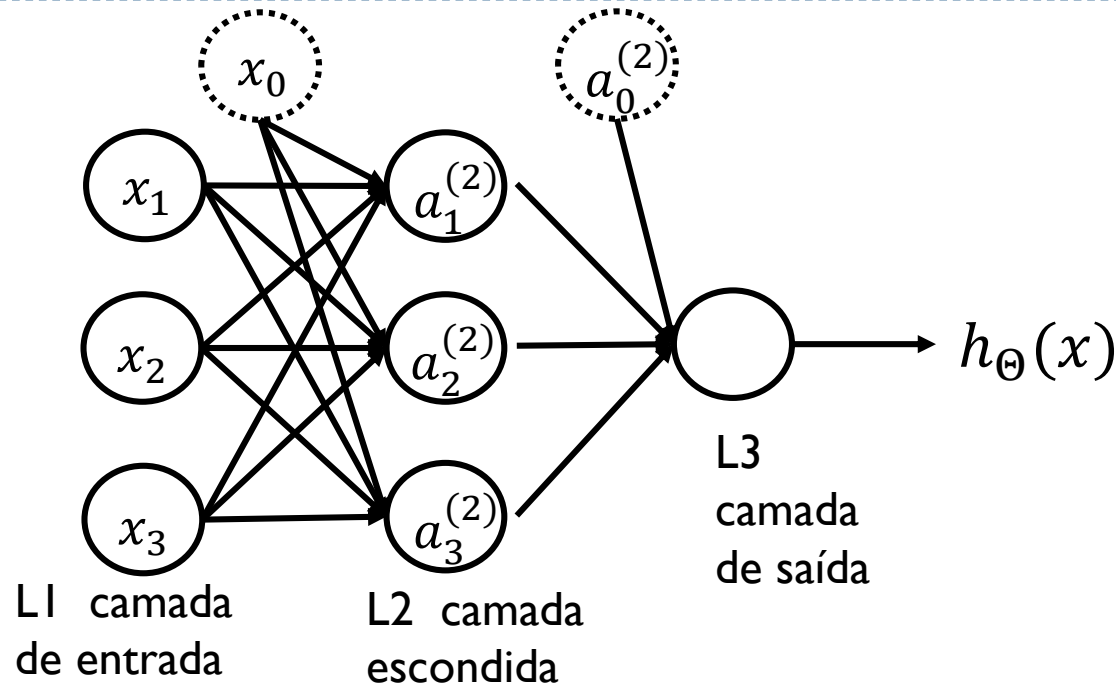


- ▶ Três unidades de entrada, três na camada escondida
 - ▶ $\Theta^{(1)} = [4 \times 3]$
- ▶ Três unidades na camada escondida, uma na saída
 - ▶ $\Theta^{(2)} = [4 \times 1]$
- ▶ Todos os nós de uma camada j são conectados aos nós da camada consecutiva $j+1$

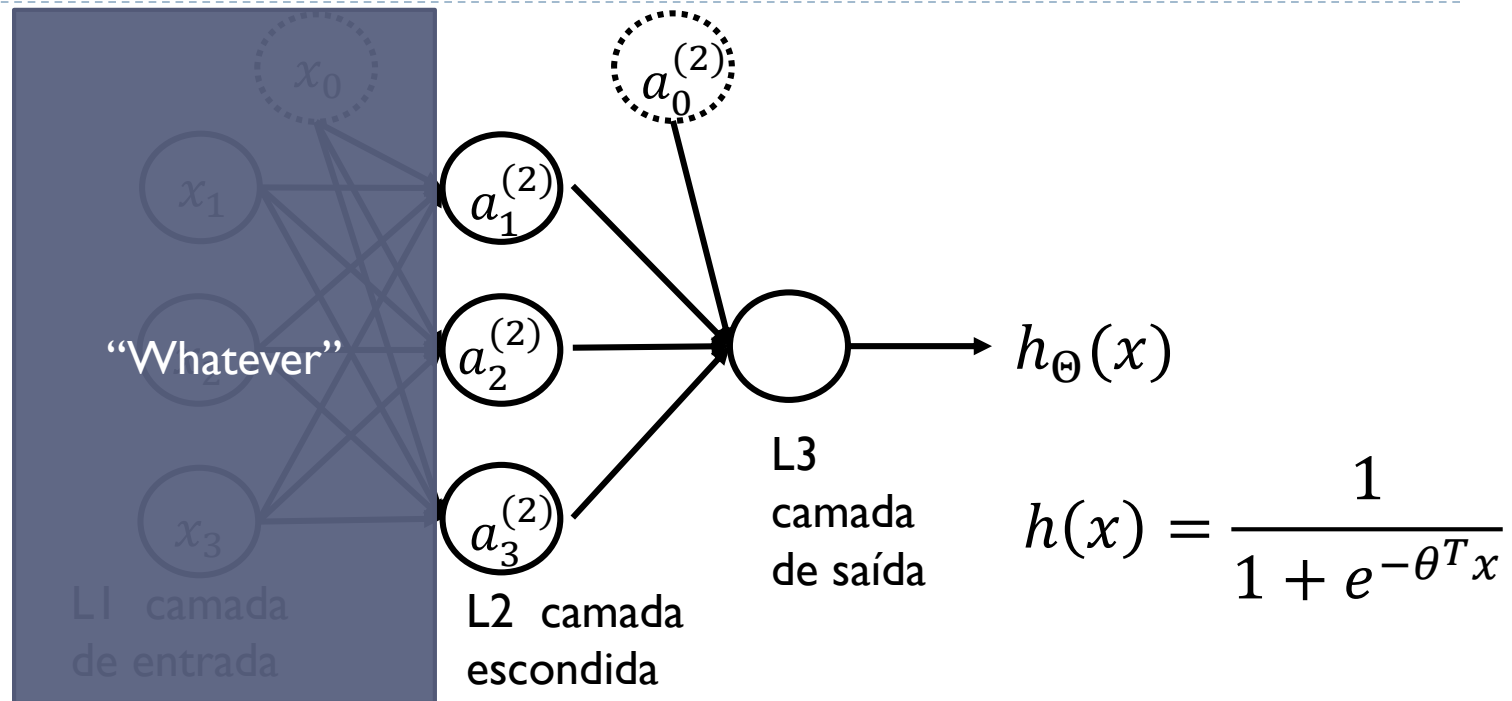
Forward propagation

- ▶ Este processo de propagação dos sinais é conhecido como *forward propagation*
 - ▶ Começa com as ativações das unidades de entrada (valores dos atributos)
 - ▶ Propaga estes sinais e calcula a ativação (saída da função sigmoide) de cada camada sequencialmente
 - ▶ *Implementation trick*: facilita a vetorização

Redes neurais aprendendo seus próprios atributos



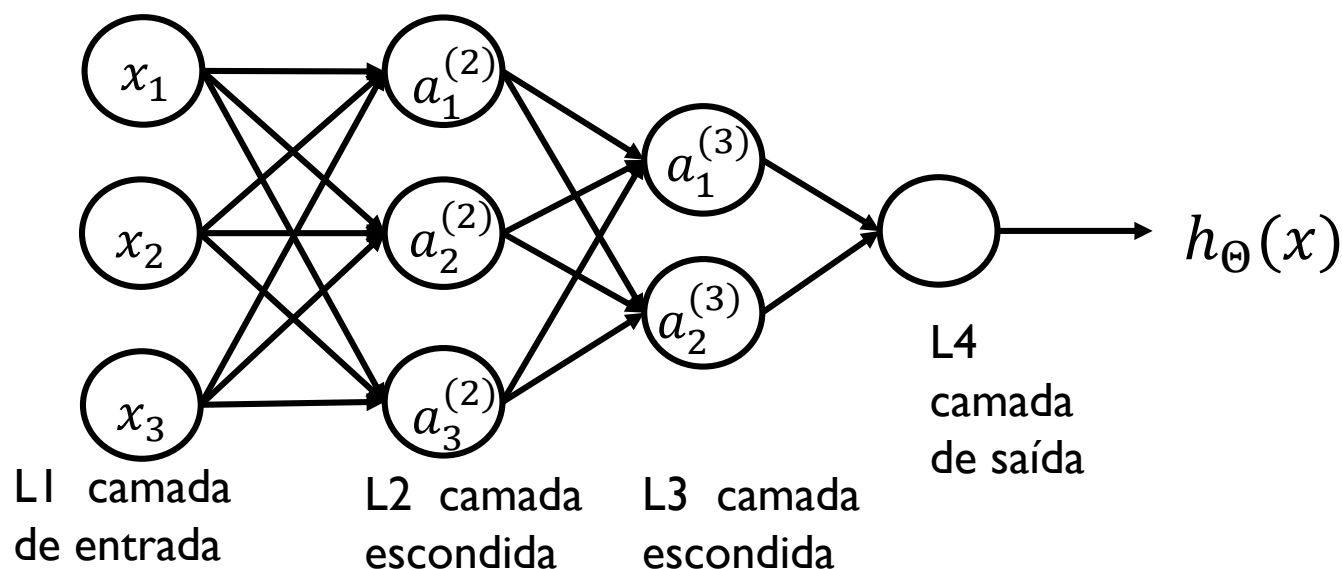
Redes neurais aprendendo seus próprios atributos



- ▶ Regressão logística em função de $a_1^{(2)}$, $a_2^{(2)}$ e $a_3^{(2)}$
 - ▶ Não são os atributos iniciais
 - ▶ São atributos aprendidos na camada anterior
 - ▶ Dependem de $\Theta^{(1)}$
 - ▶ **Classificação não linear!**

Diferentes topologias

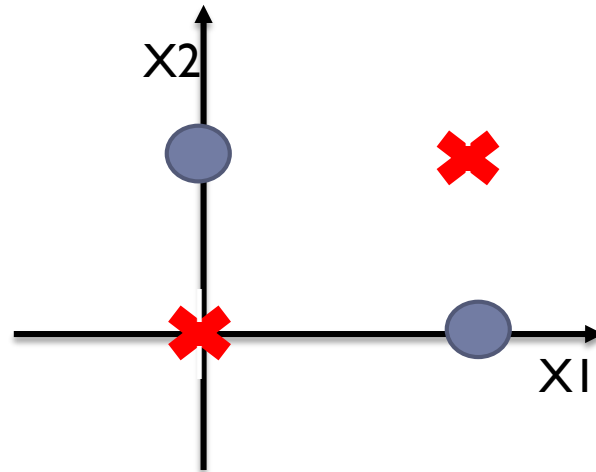
- ▶ É possível distribuir os nós em diferentes topologias
 - ▶ Diferentes números de camadas
 - ▶ Diferentes números de unidades por camada
 - ▶ Quanto maior o número de camadas / unidades, mais complexa é a hipótese aprendida



Exemplo

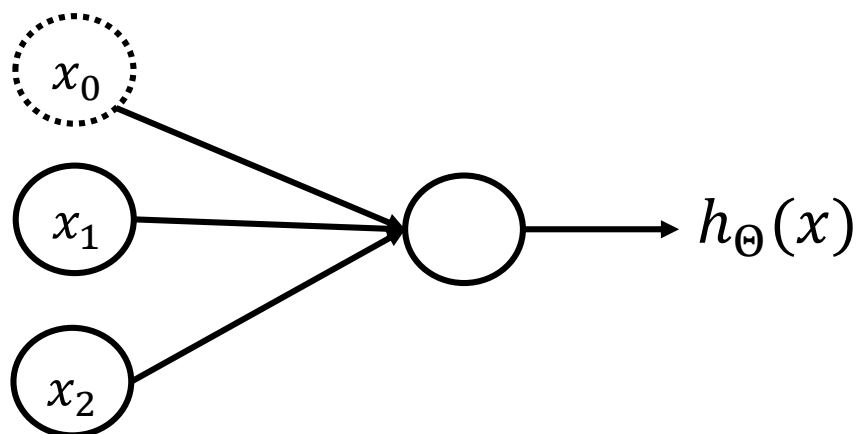
Funções complexas (não lineares)

- ▶ Função XNOR (NOT XOR)
 - ▶ Positivo quando (1,1) ou (0,0)
 - ▶ Borda de decisão não-linear



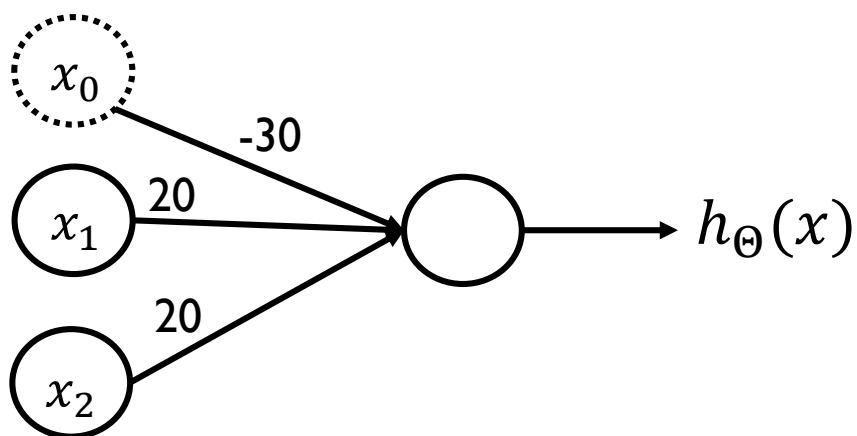
Função AND

- ▶ É possível fazer com que uma rede neural com um único neurônio compute a função AND?

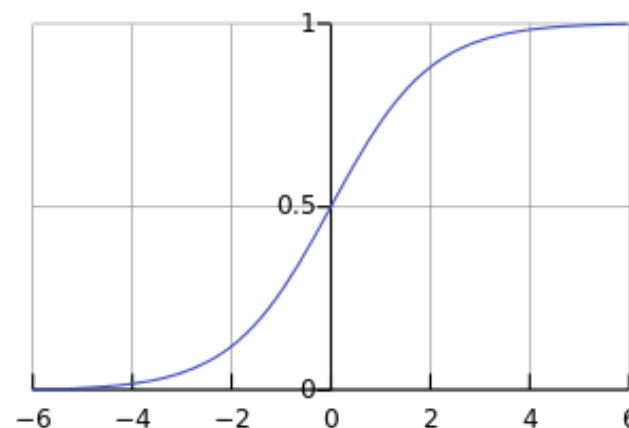


Função AND

- É possível fazer com que uma rede neural com um único neurônio compute a função AND?



$$h_{\Theta}(x) = g(-30 + 20x_1 + 20x_2)$$

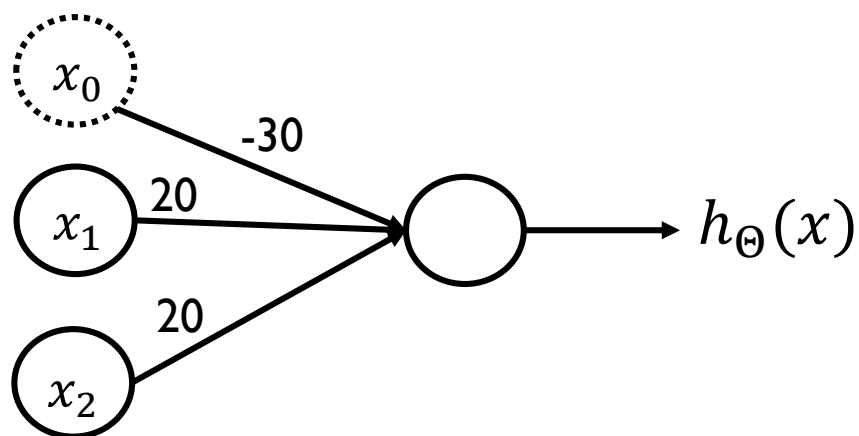


Se $z = 4,6$ $g(z) \approx 0,99$

Se $z = -4,6$ $g(z) \approx 0,01$

Função AND

- É possível fazer com que uma rede neural com um único neurônio compute a função AND?

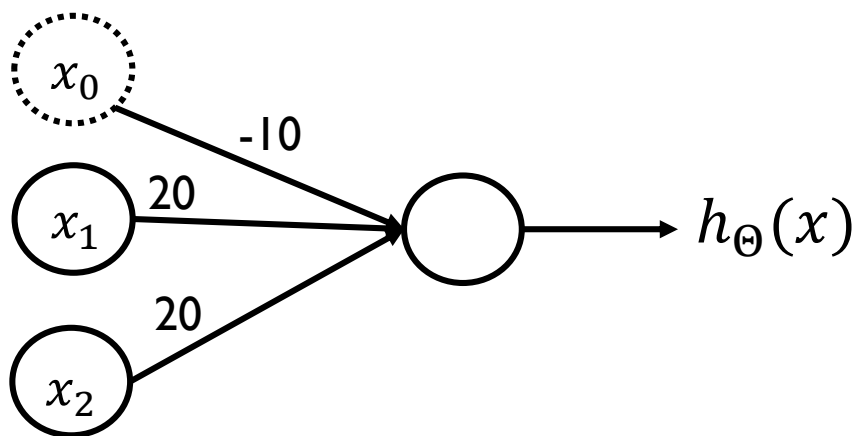


x_1	x_2	$h_{\Theta}(x)$
0	0	$g(-30) \approx 0$
0	1	$g(-10) \approx 0$
1	0	$g(-10) \approx 0$
1	1	$g(10) \approx 1$

$$h_{\Theta}(x) = g(-30 + 20x_1 + 20x_2)$$

$$h_{\Theta}(x) \approx x_1 \text{ and } x_2$$

Função OR



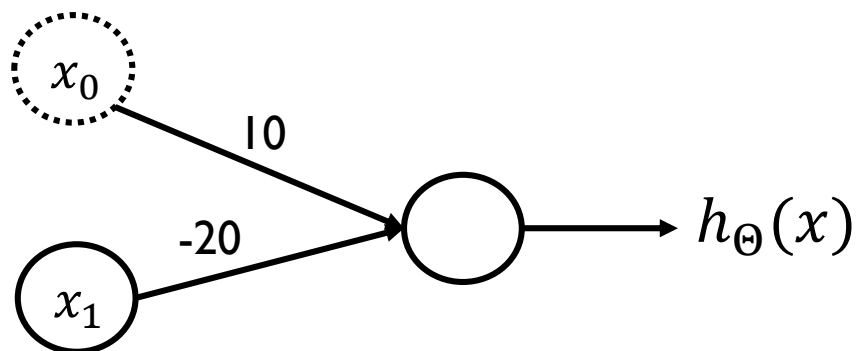
$$h_{\Theta}(x) = g(-10 + 20x_1 + 20x_2)$$

x_1	x_2	$h_{\Theta}(x)$
0	0	$g(-10) \approx 0$
0	1	$g(10) \approx 1$
1	0	$g(10) \approx 1$
1	1	$g(30) \approx 1$

$$h_{\Theta}(x) \approx x_1 \text{ or } x_2$$

Função NOT

► Negação de um valor

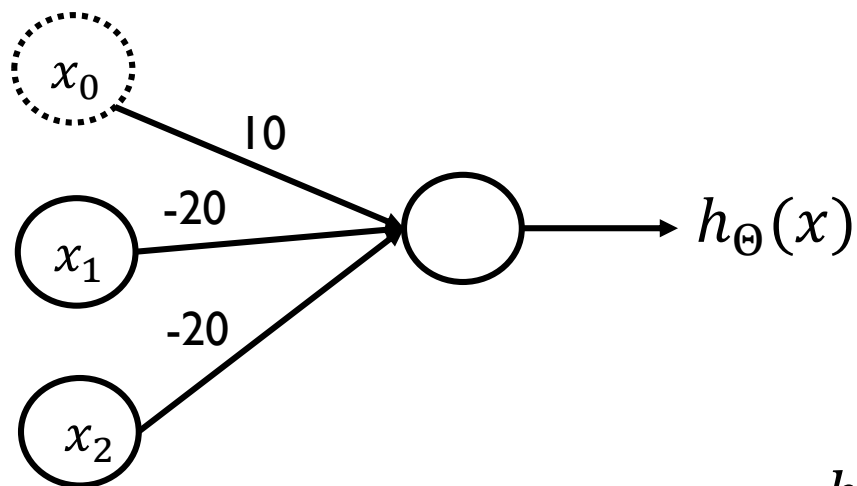


x_1	$h_{\Theta}(x)$
0	$g(10) \approx 1$
1	$g(-10) \approx 0$

$$h_{\Theta}(x) \approx !x_1$$

Função (NOT x_1) AND (NOT x_2)

- Vai ser igual a 1 somente quando $x_1 = x_2 = 0$

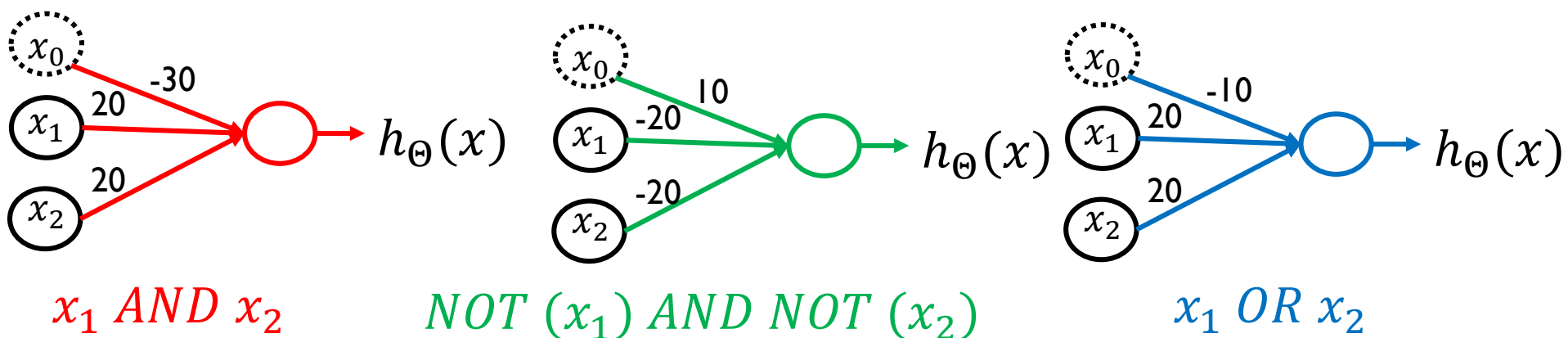


x_1	x_2	$h_{\Theta}(x)$
0	0	$g(10) \approx 1$
0	1	$g(-10) \approx 0$
1	0	$g(-10) \approx 0$
1	1	$g(-30) \approx 0$

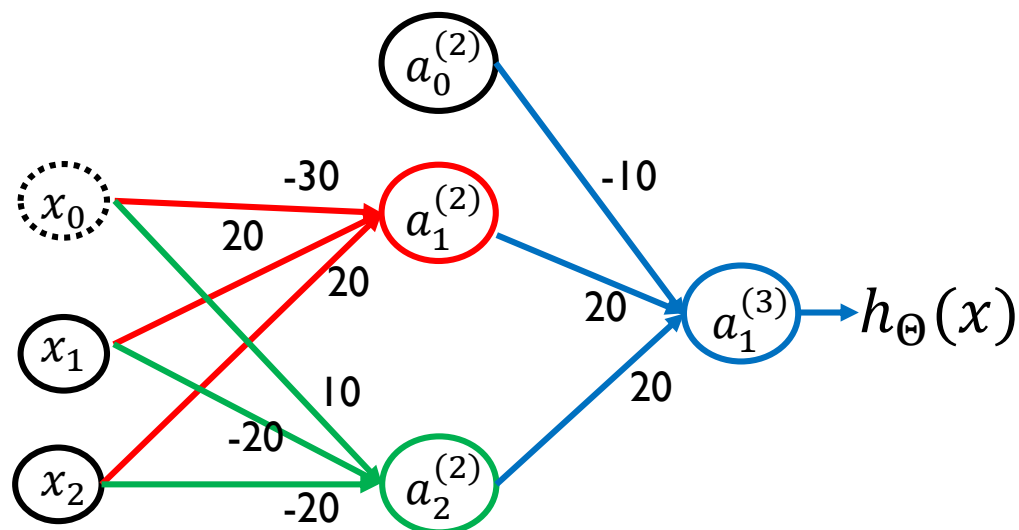
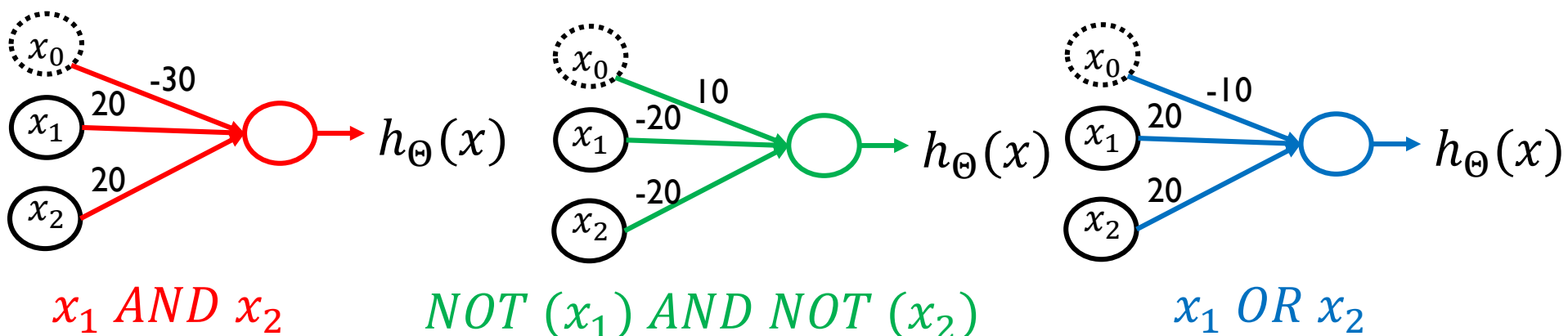
$$h_{\Theta}(x) \approx \text{NOT}(x_1) \text{ AND } \text{NOT}(x_2)$$

$$h_{\Theta}(x) = g(10 - 20x_1 - 20x_2)$$

Juntando tudo: função XNOR



Juntando tudo: função XNOR



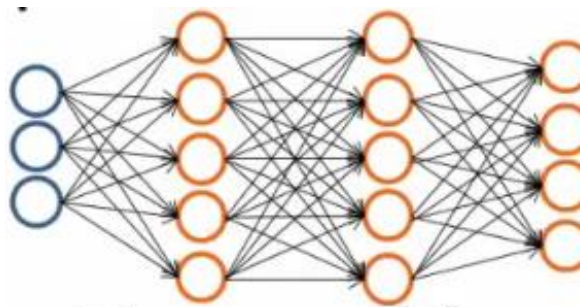
x_1	x_2	$a_1^{(2)}$	$a_2^{(2)}$	$h_{\Theta}(x)$
0	0	0	1	1
0	1	0	0	0
1	0	0	0	0
1	1	1	0	1

Classificação multiclasse

- ▶ Redes como vimos resolvem problemas binários
 - ▶ Dois valores de classe
 - ▶ Um codificado como 0, outro codificado como 1
- ▶ Entretanto, é comum encontrarmos problemas multiclasse
 - ▶ Vários valores de classe
 - ▶ Exemplo: distinguir dígitos
 - ▶ 10 possíveis classes (0 – 9)
 - ▶ Impossível classificar com um único neurônio na camada de saída

Classificação multiclasse

- ▶ Solução: utilizar abordagem um contra todos (*one vs. all*)
 - ▶ Número de neurônios na camada de saída é igual ao número de classes
 - ▶ Saída passa a ser codificada não por um valor binário, mas sim por um vetor de valores binários
 - ▶ Exemplo: uma rede para reconhecimento de pedestres, carros, motos ou caminhões
 - ▶ Vetor de 4 posições
 - $[1,0,0,0]$ = carro
 - $[0,1,0,0]$ = moto
 - $[0,0,1,0]$ = caminhão
 - $[0,0,0,1]$ = pedestre

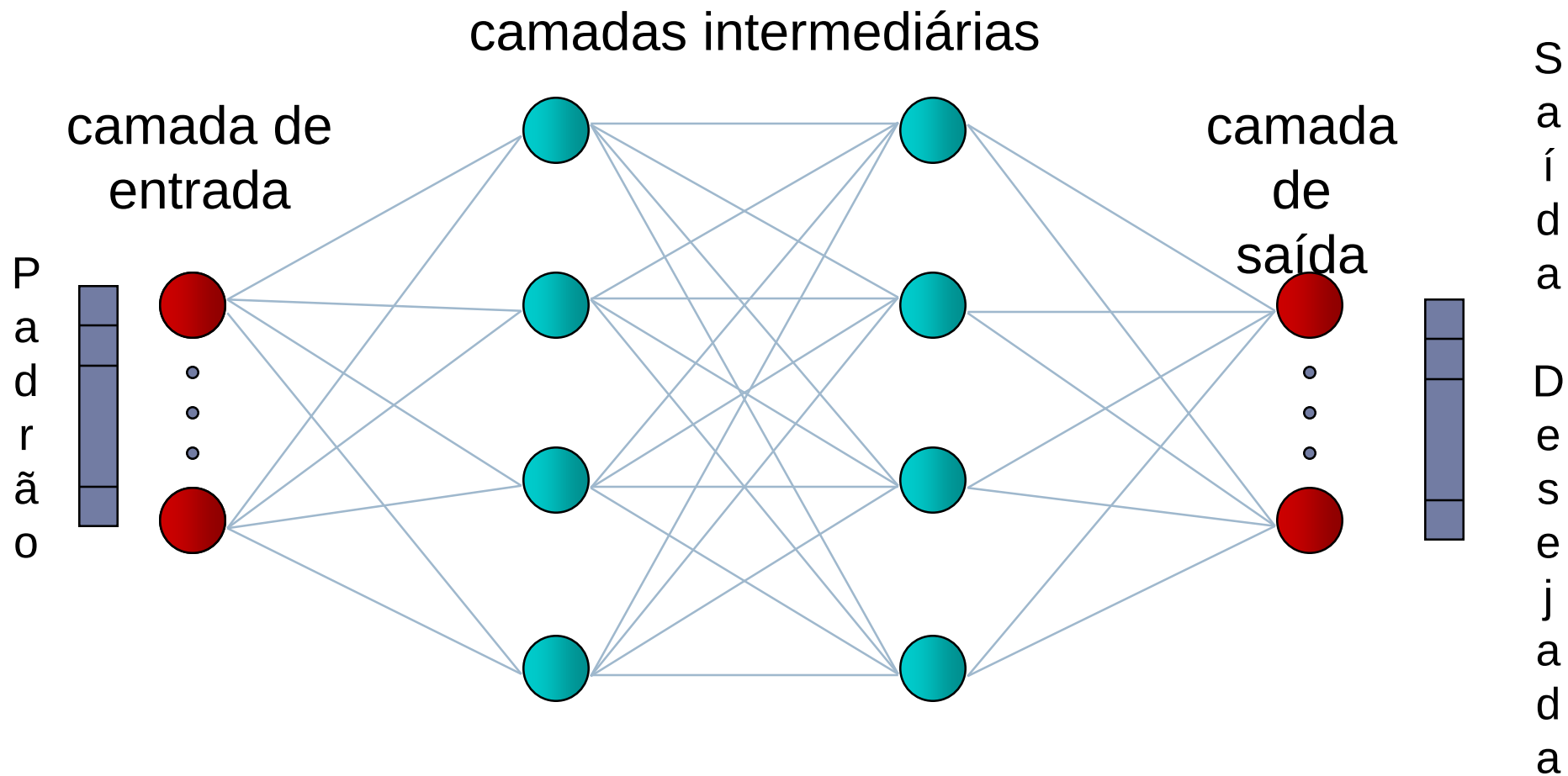


$$h_{\Theta}(x) \in \mathbb{R}^4$$

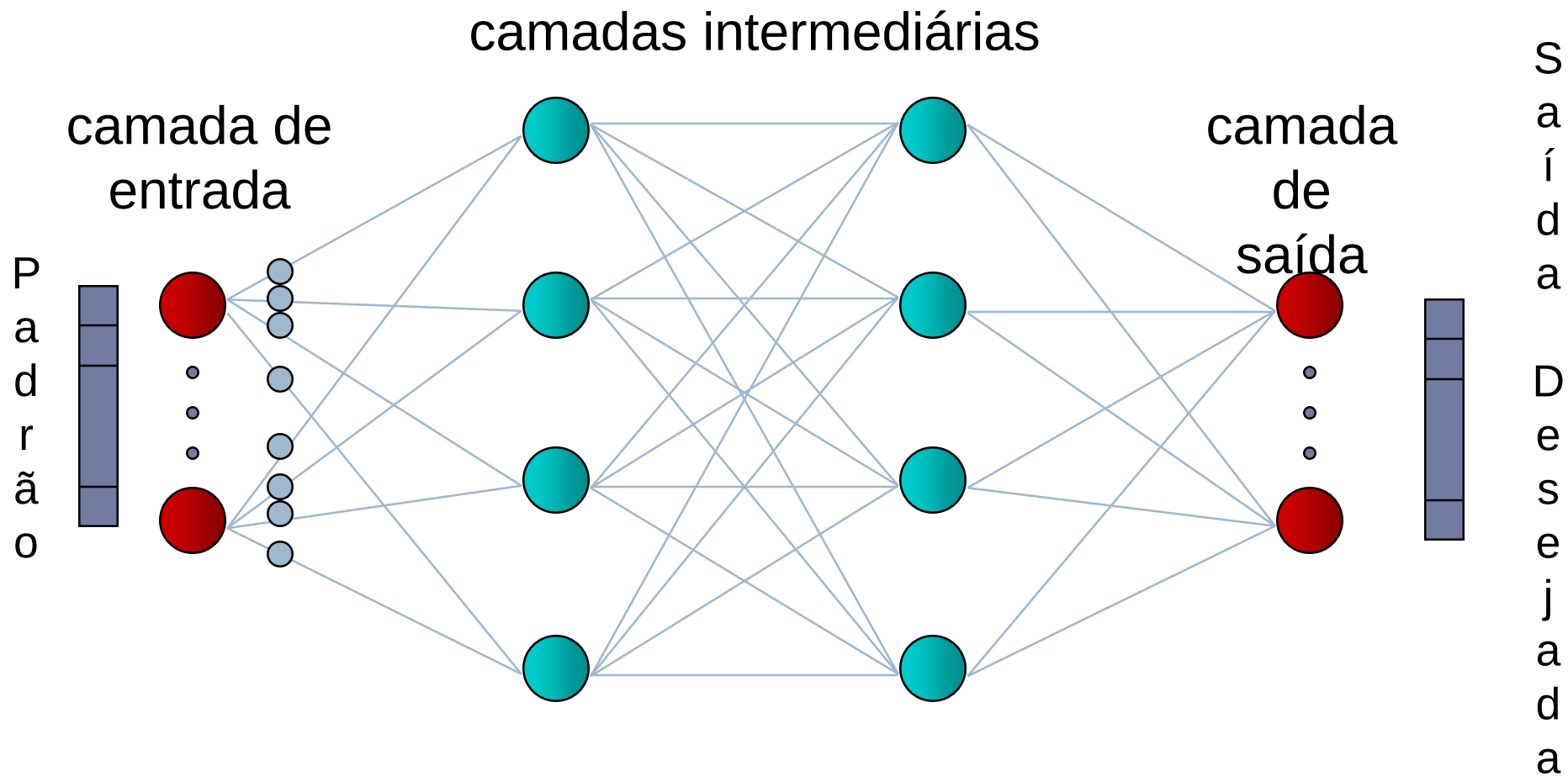
Retropropagação dos erros

- ▶ Algoritmo Backpropagation é utilizado para aprendizado das redes neurais
- ▶ Visa à minimização da função de custo
- ▶ Duas fases: *forward* e *backward*
 - ▶ Forward
 - ▶ Propagação dos sinais de ativação das unidades
 - ▶ Da camada de entrada para a camada de saída
 - ▶ Já vimos antes...
 - ▶ Backward
 - ▶ Propagação dos erros para ajustes dos pesos
 - ▶ Da camada de saída para a camada de entrada

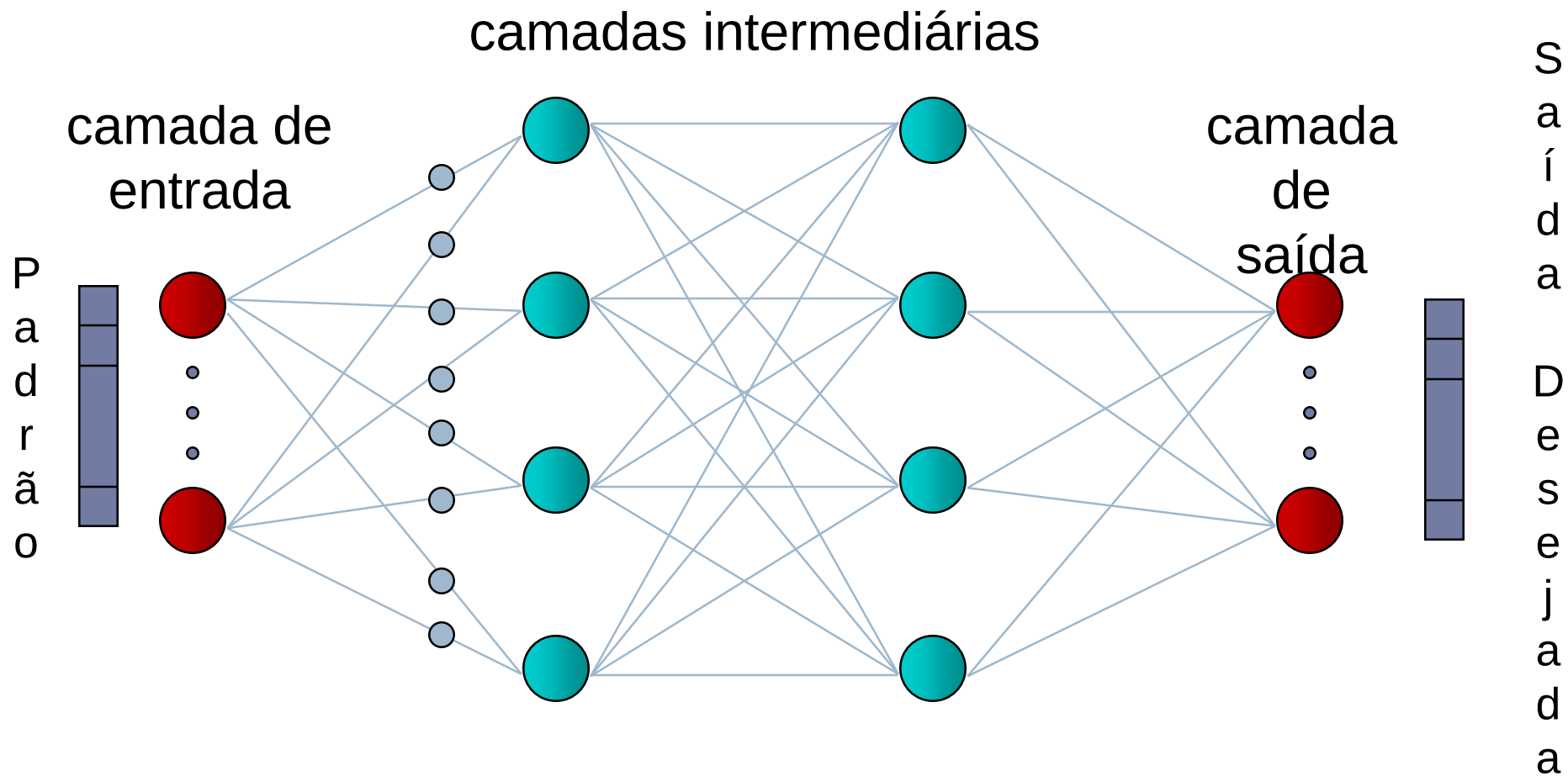
Backpropagation - Intuição



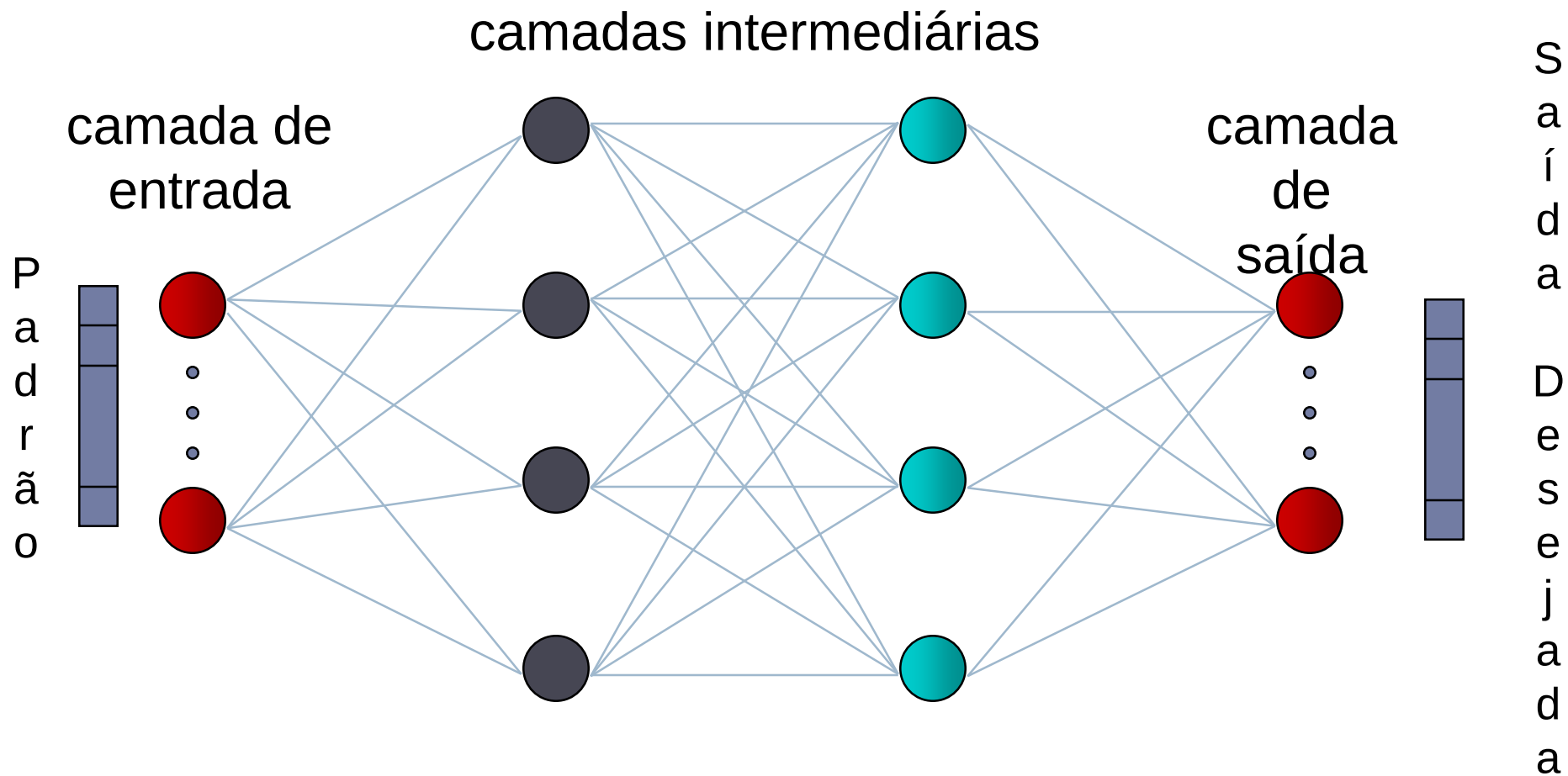
Backpropagation - Intuição



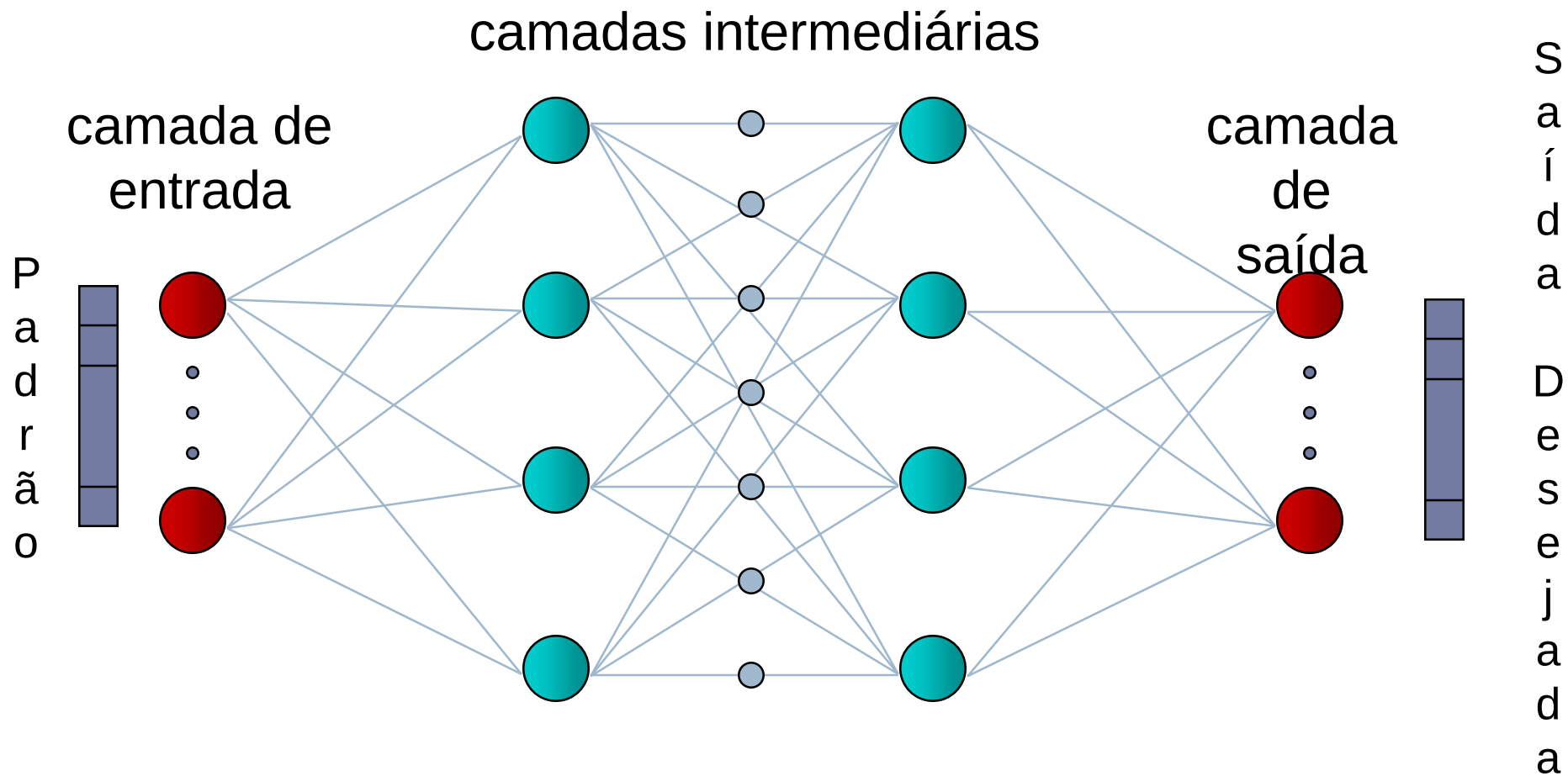
Backpropagation - Intuição



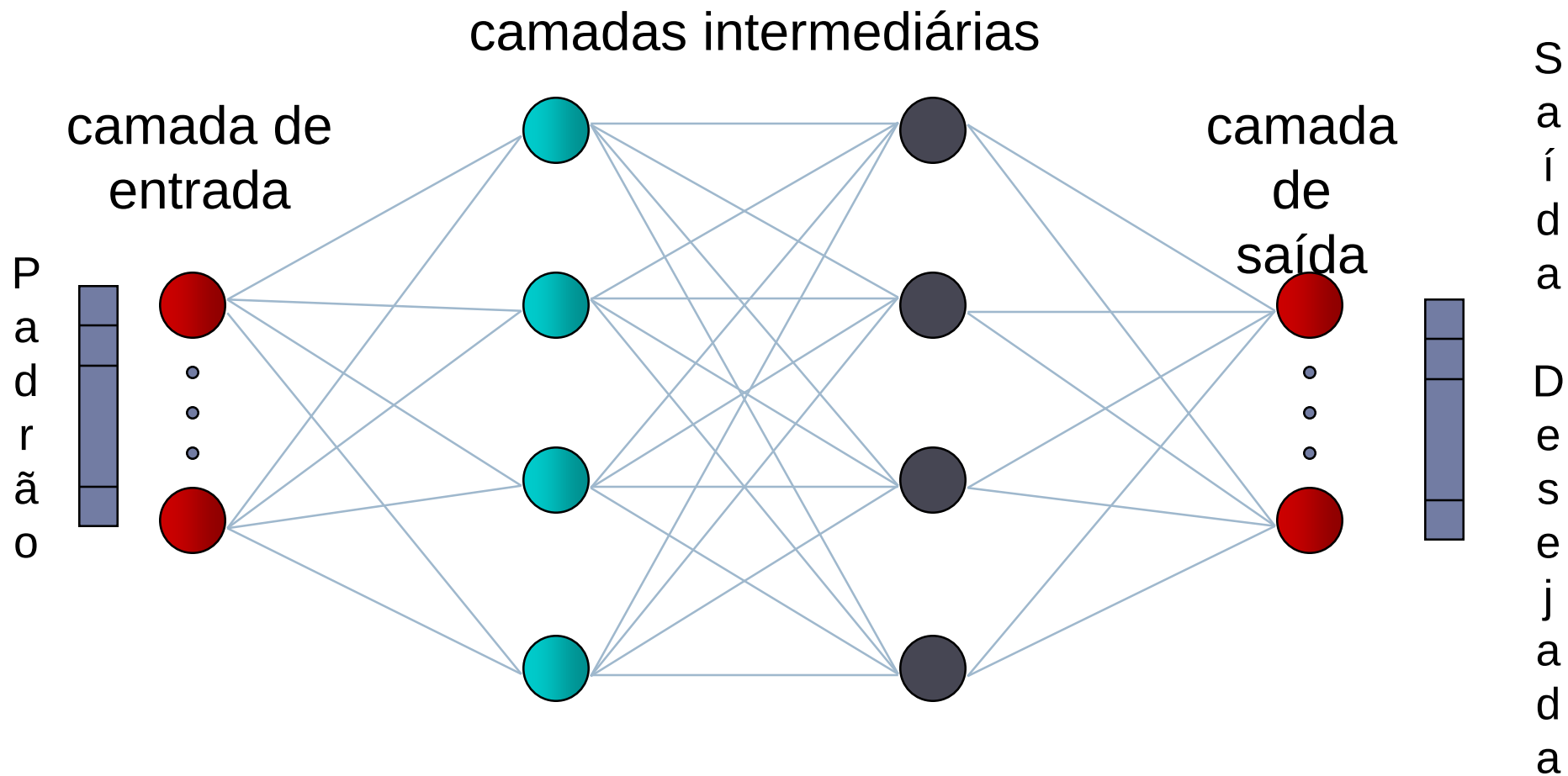
Backpropagation - Intuição



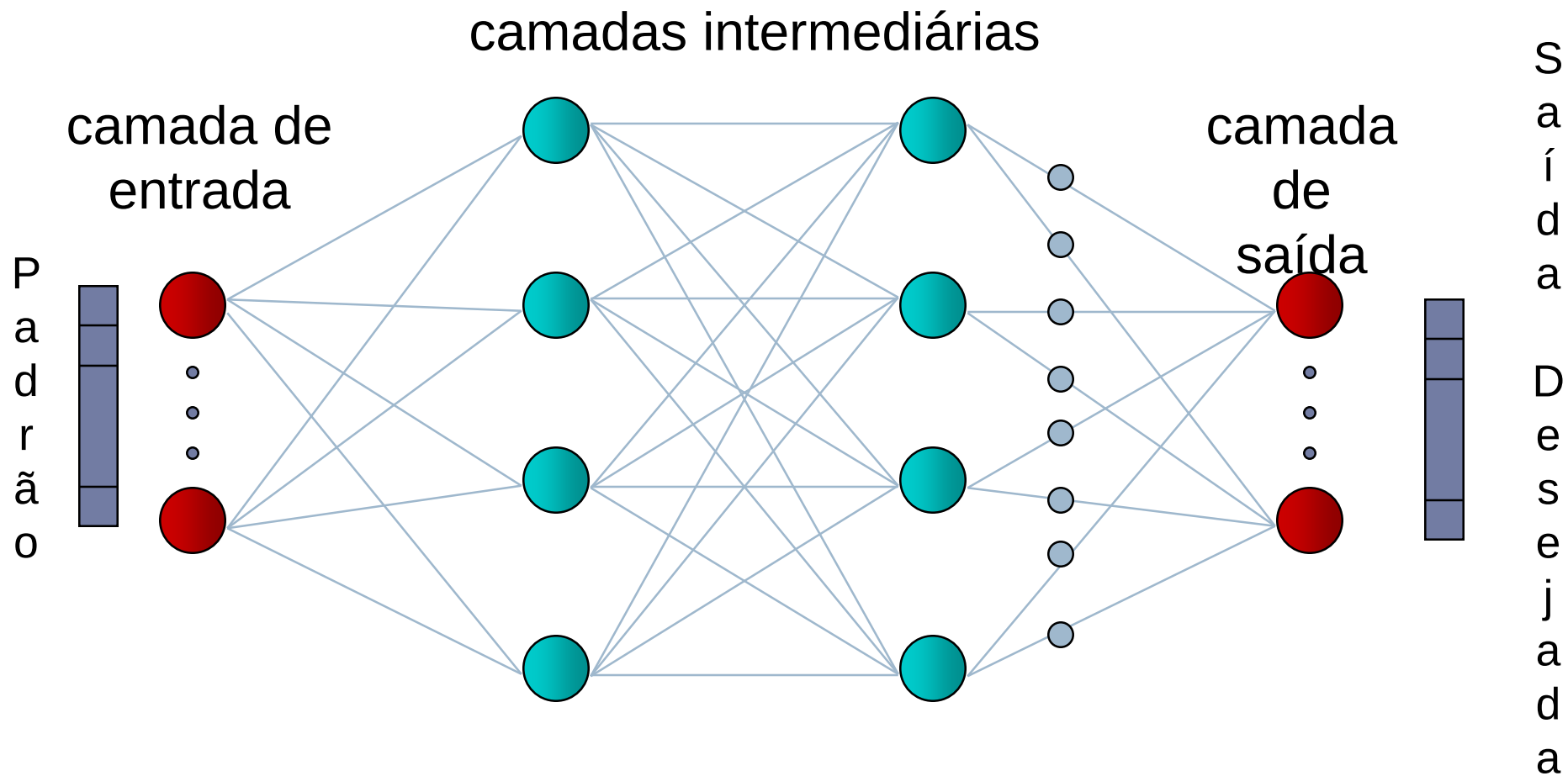
Backpropagation - Intuição



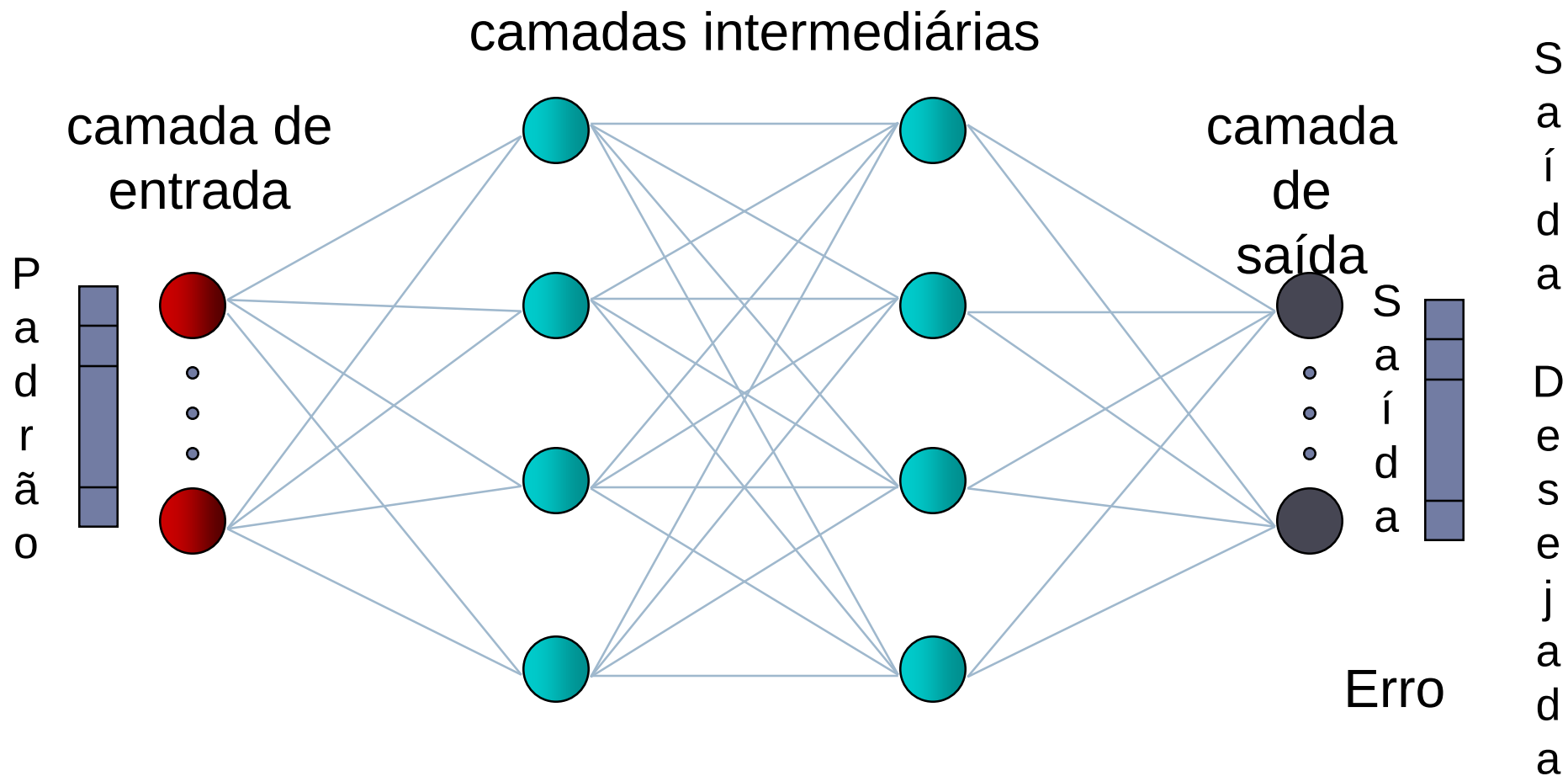
Backpropagation - Intuição



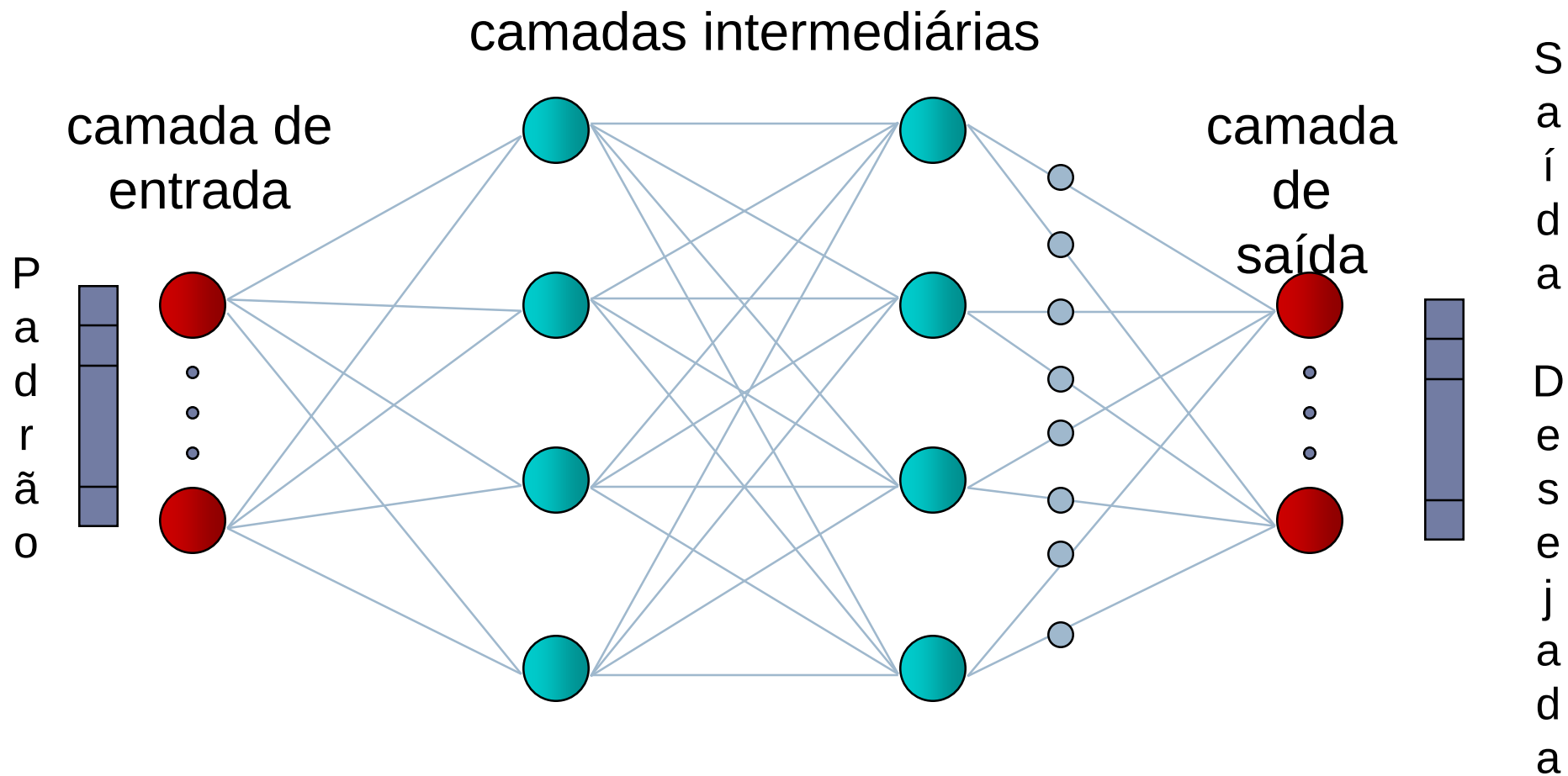
Backpropagation - Intuição



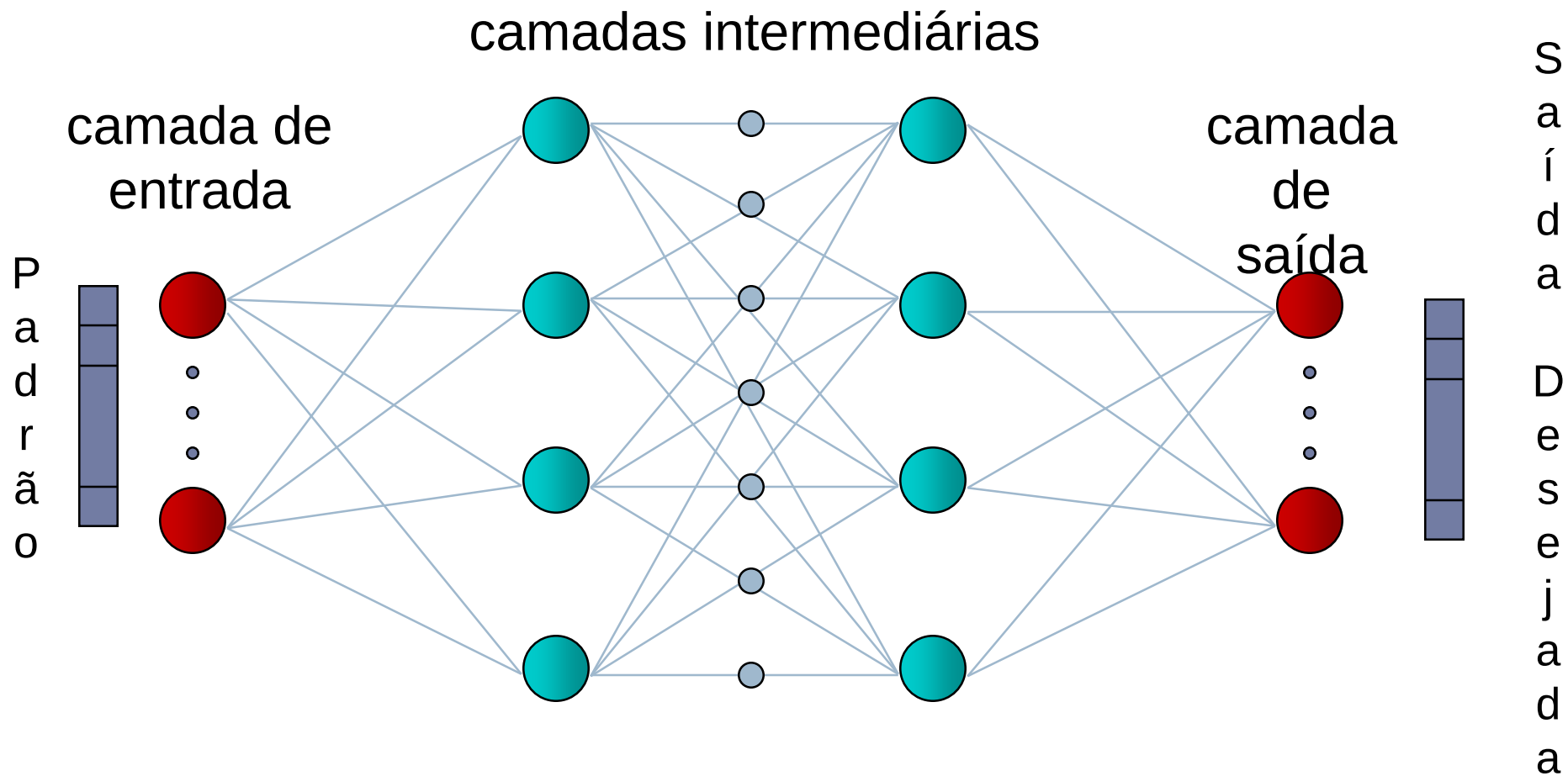
Backpropagation - Intuição



Backpropagation - Intuição



Backpropagation - Intuição



Backpropagation - Intuição

